Week 4 - Univariate Analysis, part 2

1. Lesson - None

2. Weekly graph question

Below are a histogram and boxplot representation of the same data. A pharmacy is keeping a record of the prices of the drugs that it sells, and an administrator wants to know how much the more expensive drugs tend to cost, in the context of the other prices.

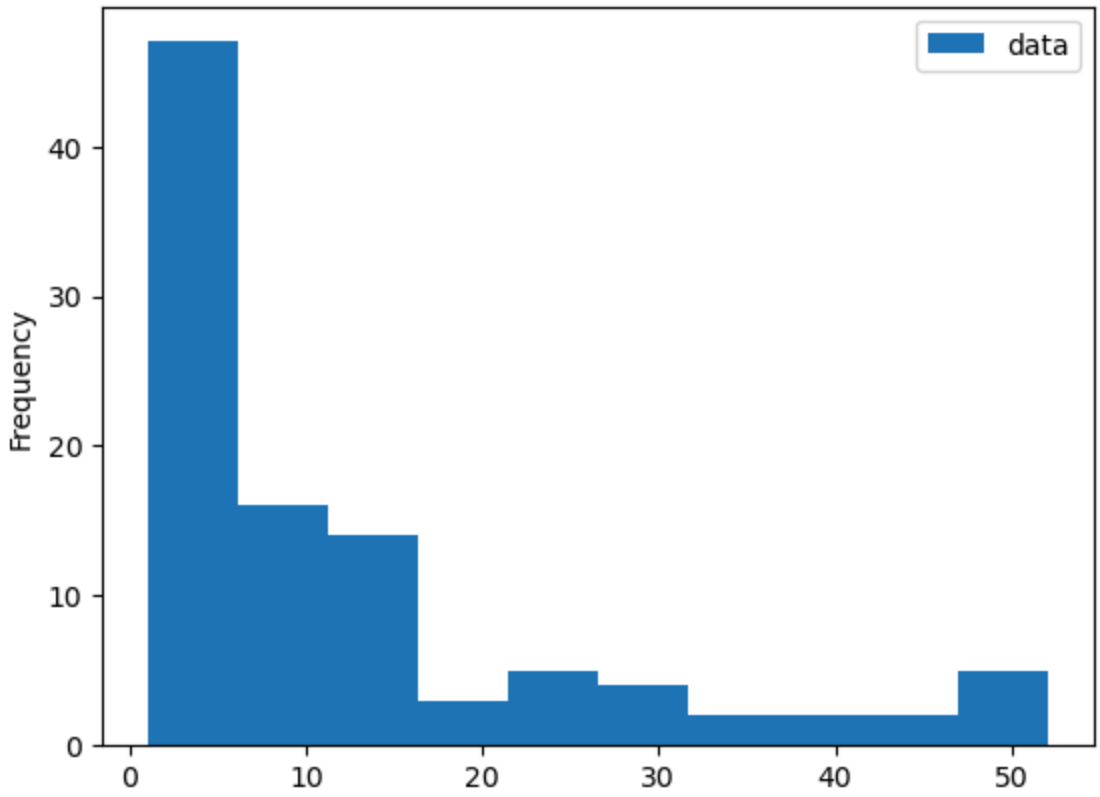
Please write a short explanation of the pros and cons of these two representations. Which would you choose? How would you modify the formatting, if at all, to make it more visually interesting, clear, or informative?

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(0)
num_data = 100
data = np.exp(np.random.uniform(size = num_data) * 4)
df = pd.DataFrame(data.T, columns = ["data"])
```

In [2]:
```
print("The 75th percentile is:", df.quantile(q = 0.75))
df.plot.hist()
```

```
The 75th percentile is: data    15.457656
Name: 0.75, dtype: float64
```

Out[2]:
```
<Axes: ylabel='Frequency'>
```



Neither of the plots are easily consumable in that they do not have any contextual information (such as what the data actually is representing, what each axis represents, etc.).

The Histogram

- Pros:
  1. The histogram is easily readable in that it is in a format we are more used to than others
- Cons:
  1. The histogram is vertical making us do the eye zigzag instead of being horizontal, which would be easier to read
  2. As in a previous example, the legend is next to useless in that all it says is that this is data
  3. The y-axis label is similarly useless in that frequency has very little meaning in this context
  4. There probably should be either more bins and/or more space between the bins

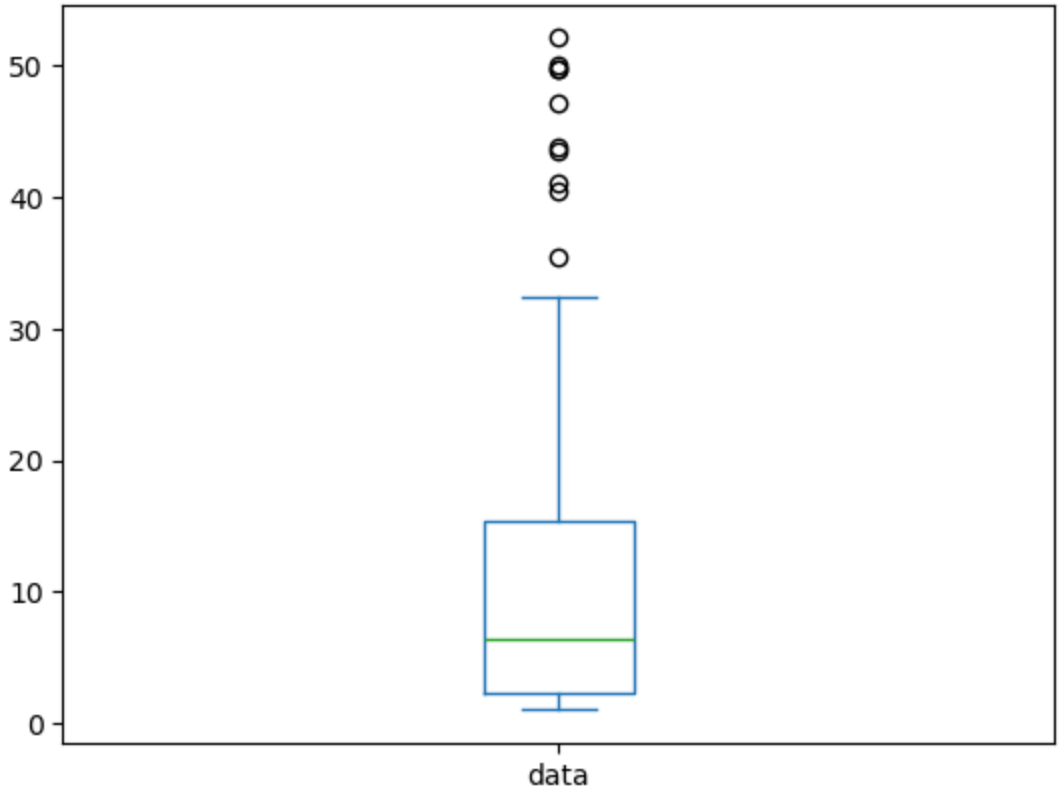
The boxplot:

- Pros:
  1. One can more easily see outliers using the boxplot
  2. The green line is helpful, if one knows what it means
  3. The whiskers help to visualize the min and max, if one knows what they mean

- Cons:
  1. Although the boxplot has many features that could be informative, without labelling there is very little one can immediately consume from this plot
  2. The boxplot does not have labels either
  3. The whiskers are not explained
  4. Outliers are not explained
  5. The green line, although present, is not explained

In [3]:
```
df.plot.box()
```

Out[3]: <Axes: >



Which to choose?

If I were to choose from the graphs depicted above, I would choose the histogram - with certain provisions.

- The changes I suggest in the formatting changes below would need to be instituted
  - The changes must include:
    1. A title which describes the data depicted
    2. Axis labels that clearly inform the consumer of what is being graphed
    3. a separation of the bins so that it is more clear

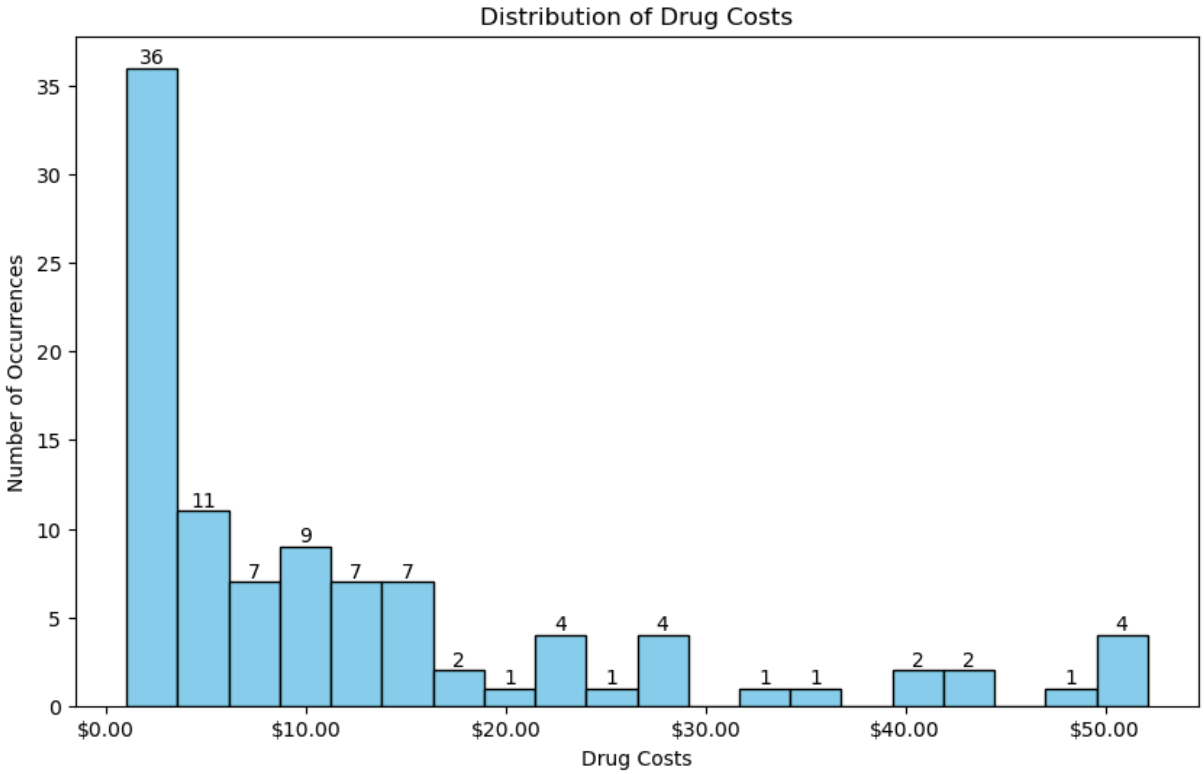
Formatting Change, if any.

I would change the histogram plot code to include

- The title for the plot
- Descriptive labels for the x and y axes
- Either a legend that describes the data being displayed, or no legend at all
- Data labels would help, especially in this context

In [4]:
```
# I import FuncFormatter from matplotlib.ticker to format the x-axis of the histogr
from matplotlib.ticker import FuncFormatter
# in the print statement, one immediately wants to know what it means. I also added
print(f"The 75th percentile of all drug costs is: ${round(df.quantile(q = 0.75).ite
print("which means that 75% of the drug costs are below this value and 25% are abov
# the plot has been changed to include some styling and labels to make it more info

ax = df.plot.hist(legend = False, bins = 20, figsize = (10, 6), color = "skyblue",
ax.xaxis.set_major_formatter(FuncFormatter(lambda x, p: f'${x:.2f}'))

# code to add data labels on top of each bar
for p in ax.patches:
    height = p.get_height()
    if height > 0:
        ax.text(p.get_x() + p.get_width()/2., height,
                f'{int(height)}',
                ha='center', va='bottom')
```

```
The 75th percentile of all drug costs is: $15.46
which means that 75% of the drug costs are below this value and 25% are above it.
```



3. Homework - working on your datasets

This week, you will do the same types of exercises as last week, but you should use your chosen datasets that someone in your class found last semester. (They likely will not be the particular datasets that you found yourself.)

Here are some types of analysis you can do:

Read in dataframe

In [5]:
```
dcd25 = pd.read_csv("..\dcd_2025.tsv", sep = "\t")
dcd25_copy = dcd25.copy()
```

Functions

In [6]:
```
def find_missing(df):
    missing_summary = pd.DataFrame({
    'Column': df.columns,
    'Missing_Count': df.isnull().sum().values,
    'Missing_Percent': (df.isnull().sum() / len(df) * 100).values
    })
    missing_summary = missing_summary.sort_values('Missing_Percent', ascending=Fals
    print(missing_summary)

    return missing_summary
```

In [7]:
```
def show_missing_columns(df, lower_bound, upper_bound):
    missing_percent = (df.isnull().sum() / len(df)) * 100
    filtered_missing = missing_percent[(missing_percent > lower_bound) & (missing_p
    count = len(filtered_missing)

    # Generate markdown table
    table_df = filtered_missing.reset_index()
    table_df.columns = ['Column', 'Missing %']
    table_df['Missing %'] = table_df['Missing %'].round(2)
    print(f"Count: \033[1m{count}\033[0m columns with missing values between {lower
    print(table_df.to_markdown(index=False))
    print('\n')

    return filtered_missing, count
```

In [8]:
```
dcd25summary = find_missing(dcd25)
```

```
         Column  Missing_Count  Missing_Percent
25  Unnamed: 25         171004            100.0
57  Unnamed: 57         171004            100.0
26  Unnamed: 26         171004            100.0
27  Unnamed: 27         171004            100.0
32  Unnamed: 32         171004            100.0
..          ...            ...              ...
9        enroll              0              0.0
13        amind              0              0.0
16         hisp              0              0.0
3    state_name              0              0.0
2      district              0              0.0

[65 rows x 3 columns]
```

In [9]:
```
dcd25_Percent95, dcd25_percent95_count = show_missing_columns(dcd25, 95, 100)
```

```
Count: 45 columns with missing values between 95% and 100%:
| Column      |   Missing % |
|:------------|------------:|
| Unnamed: 20 |         100 |
| Unnamed: 21 |         100 |
| Unnamed: 22 |         100 |
| Unnamed: 23 |         100 |
| Unnamed: 24 |         100 |
| Unnamed: 25 |         100 |
| Unnamed: 26 |         100 |
| Unnamed: 27 |         100 |
| Unnamed: 28 |         100 |
| Unnamed: 29 |         100 |
| Unnamed: 30 |         100 |
| Unnamed: 31 |         100 |
| Unnamed: 32 |         100 |
| Unnamed: 33 |         100 |
| Unnamed: 34 |         100 |
| Unnamed: 35 |         100 |
| Unnamed: 36 |         100 |
| Unnamed: 37 |         100 |
| Unnamed: 38 |         100 |
| Unnamed: 39 |         100 |
| Unnamed: 40 |         100 |
| Unnamed: 41 |         100 |
| Unnamed: 42 |         100 |
| Unnamed: 43 |         100 |
| Unnamed: 44 |         100 |
| Unnamed: 45 |         100 |
| Unnamed: 46 |         100 |
| Unnamed: 47 |         100 |
| Unnamed: 48 |         100 |
| Unnamed: 49 |         100 |
| Unnamed: 50 |         100 |
| Unnamed: 51 |         100 |
| Unnamed: 52 |         100 |
| Unnamed: 53 |         100 |
| Unnamed: 54 |         100 |
| Unnamed: 55 |         100 |
| Unnamed: 56 |         100 |
| Unnamed: 57 |         100 |
| Unnamed: 58 |         100 |
| Unnamed: 59 |         100 |
| Unnamed: 60 |         100 |
| Unnamed: 61 |         100 |
| Unnamed: 62 |         100 |
| Unnamed: 63 |         100 |
| Unnamed: 64 |         100 |
```

In [10]:
```
dcd25_new = dcd25.drop(columns=dcd25_Percent95.index.tolist())
dcd25_new.sample(10)
```

Out[10]:

	year	leaid	district	state_name	stabbr	ppcstot	predcost	fundinggap
52162	2018	2506210	HOLLAND	Massachusetts	MA	14506	10201	4305
100044	2014	1924960	RUDD-ROCKFORD-MARBLE RK COMM SCHOOL DISTRICT	Iowa	IA	10825	9507	1318
45406	2019	4218390	PALMYRA AREA SD	Pennsylvania	PA	11540	7661	3879
147966	2010	1302550	GWINNETT COUNTY	Georgia	GA	9278	11514	-2236
120089	2013	4843860	VALLEY VIEW ISD	Texas	TX	7948	10387	-2439
154116	2010	3805160	DIVIDE COUNTY 1	North Dakota	ND	14654	9826	4828
72602	2016	601860	ALBANY CITY UNIFIED	California	CA	12572	9338	3234
127681	2012	3100018	WAUNETA-PALISADE PUBLIC SCHS	Nebraska	NE	16651	13062	3589
64207	2017	2505280	GLOUCESTER	Massachusetts	MA	17395	10450	6945
84641	2015	502250	ALMA SCHOOL DISTRICT	Arkansas	AR	8846	8599	247

In [11]:

```
# rename columns in dcd25_new for clarity
dcd25_renamed = dcd25_new.rename(columns={'predcost': 'req_spending', 'ppcstot': 'a
dcd25_renamed.sample(10)
```

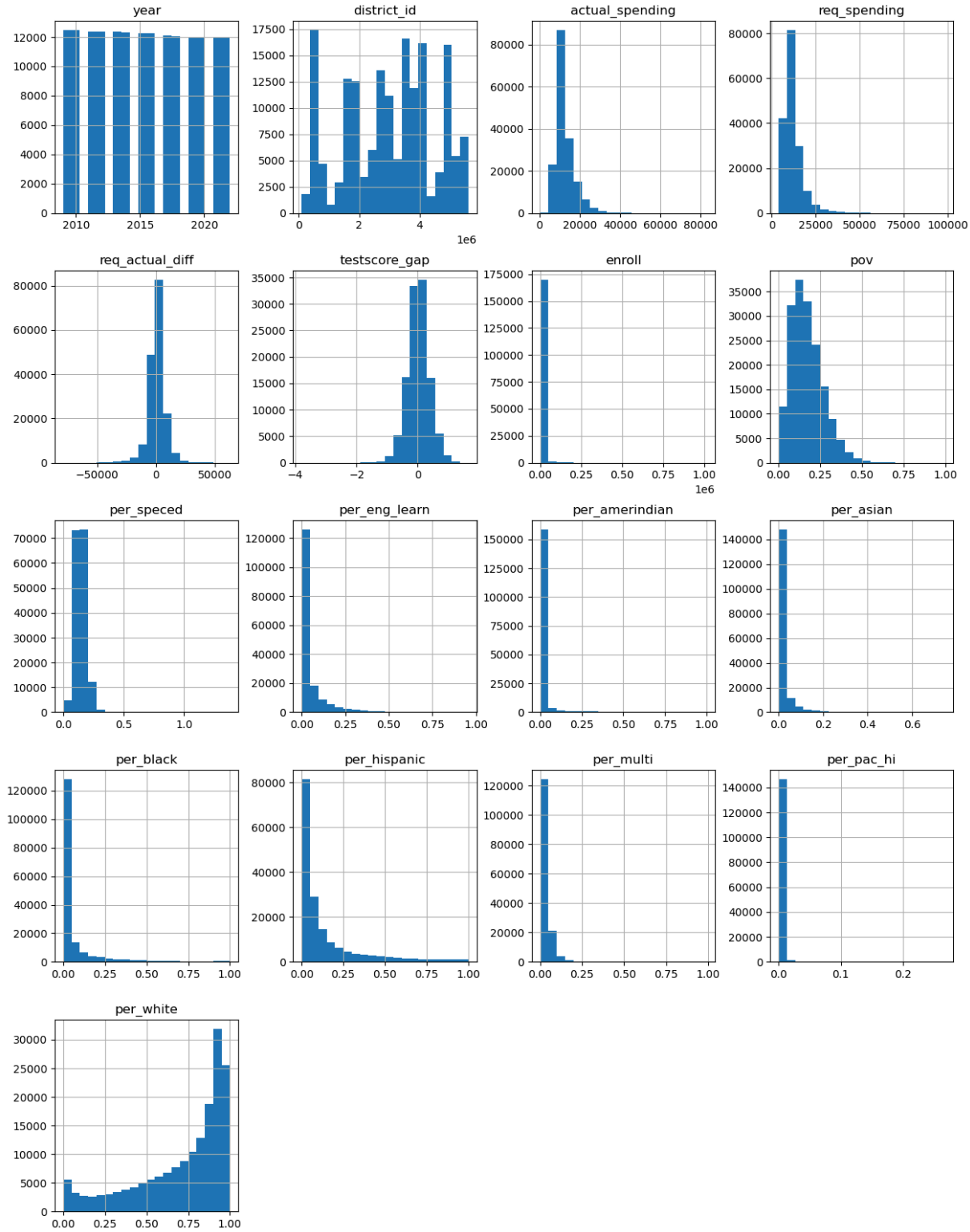
Out[11]:

	year	district_id	district	state_name	stabbr	actual_spending	req_spending
162575	2009	2200180	BEAUREGARD PARISH	Louisiana	LA	9114	8259
14214	2021	1713370	EDGAR COUNTY CUD 6	Illinois	IL	16240	19232
121262	2013	5602830	FREMONT COUNTY SCHOOL DISTRICT # 6	Wyoming	WY	21446	9317
153820	2010	3625020	ROSCOE CENTRAL SCHOOL DISTRICT	New York	NY	27520	9947
57644	2018	4400420	FOSTER-GLOCESTER	Rhode Island	RI	17337	8938
161967	2009	1920100	MURRAY COMM SCHOOL DISTRICT	Iowa	IA	9066	8605
27609	2020	2011790	SOUTH HAVEN	Kansas	KS	15330	16057
42977	2019	3604620	BERLIN CENTRAL SCHOOL DISTRICT	New York	NY	26103	11086
25654	2020	1200960	JACKSON	Florida	FL	10204	15132
66910	2017	3416290	TRENTON PUBLIC SCHOOL DISTRICT	New Jersey	NJ	16034	23558

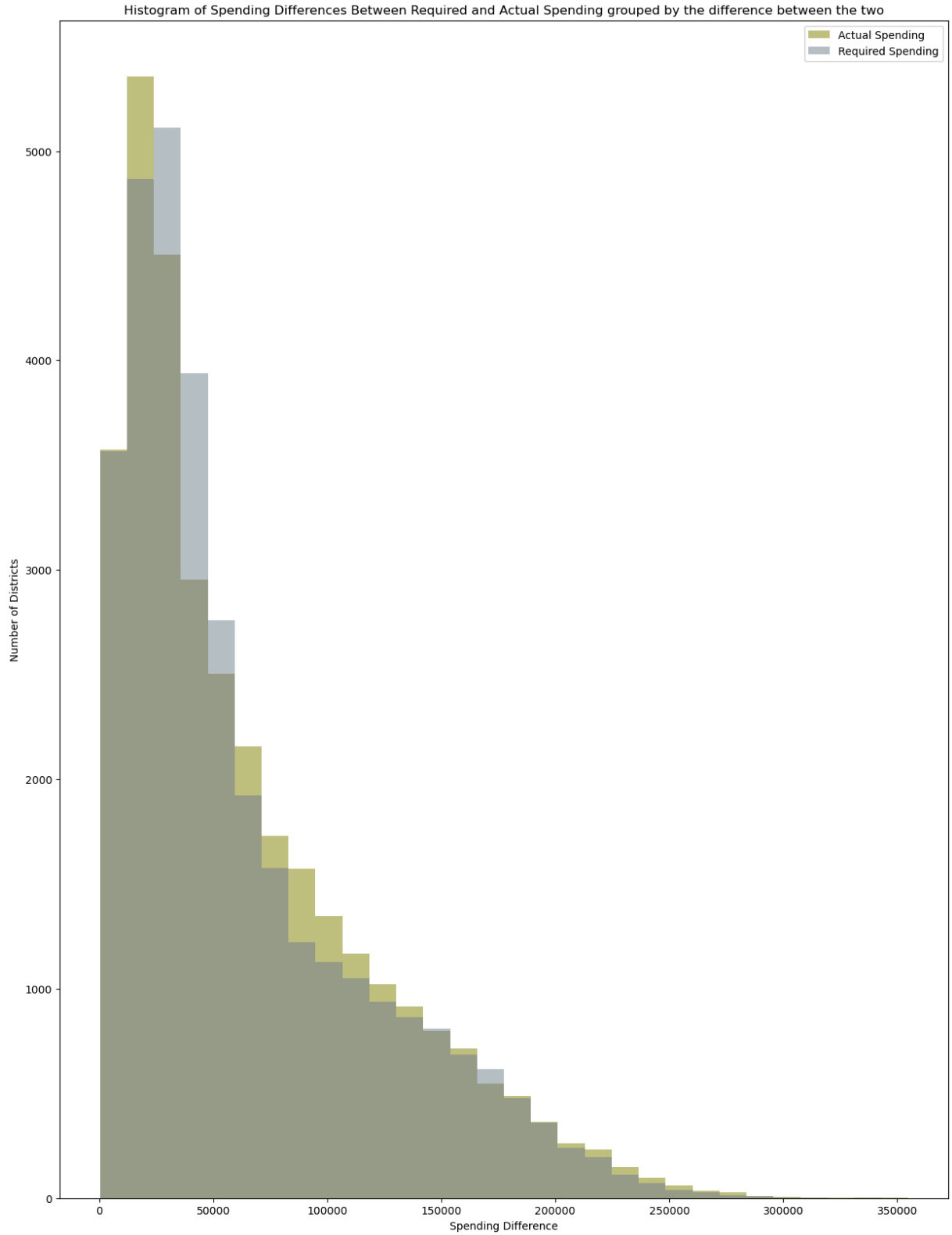
Plots of dataframe

Draw grouped histograms. For instance, if you have tree heights for both maple and oak trees, you could draw histograms for both.

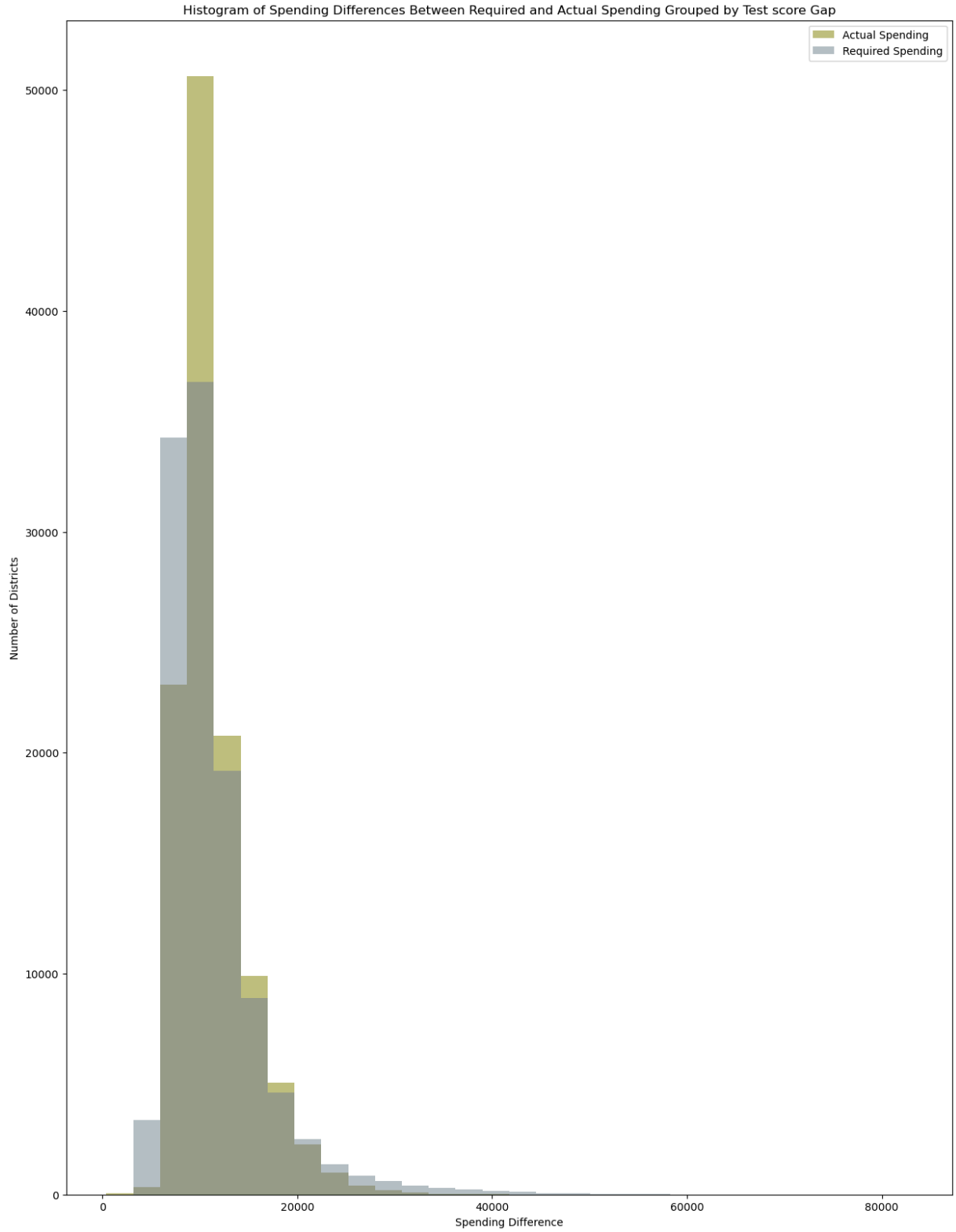
In [12]:
```
dcd25_renamed.hist(figsize=(15, 20), bins=20);
```



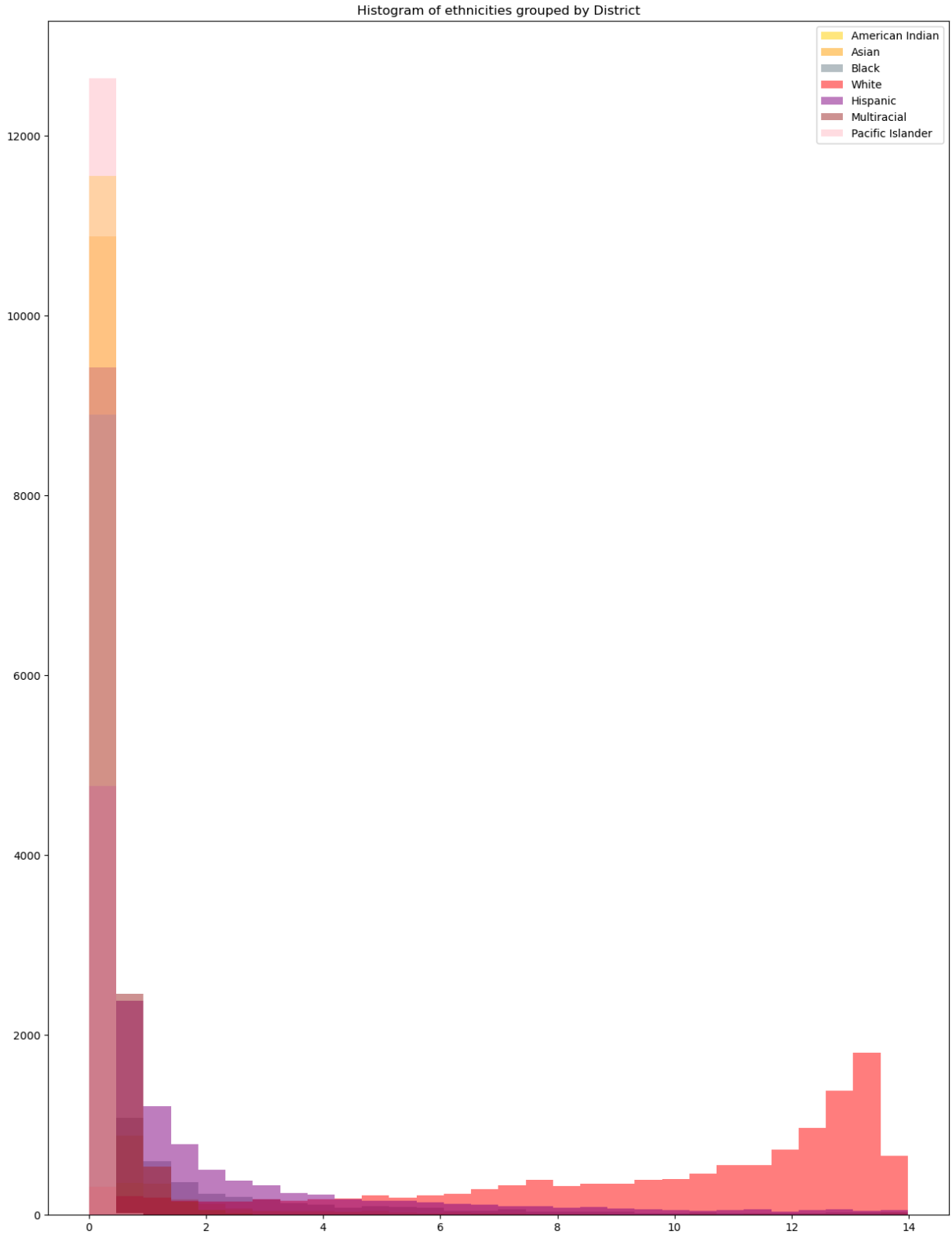
In [13]:
```
districtID_actual_spending = dcd25_renamed.groupby('req_actual_diff')[['actual_spen
ax = districtID_actual_spending.plot(kind='hist', legend=True, figsize=(15, 20), bi
ax.legend(['Actual Spending', 'Required Spending']);
```



In [14]:
```
districtID_spending_score_gap = dcd25_renamed.groupby('testscore_gap')[['actual_spe
ax = districtID_spending_score_gap.plot(kind='hist', legend=True, figsize=(15, 20),
ax.legend(['Actual Spending', 'Required Spending']);
```



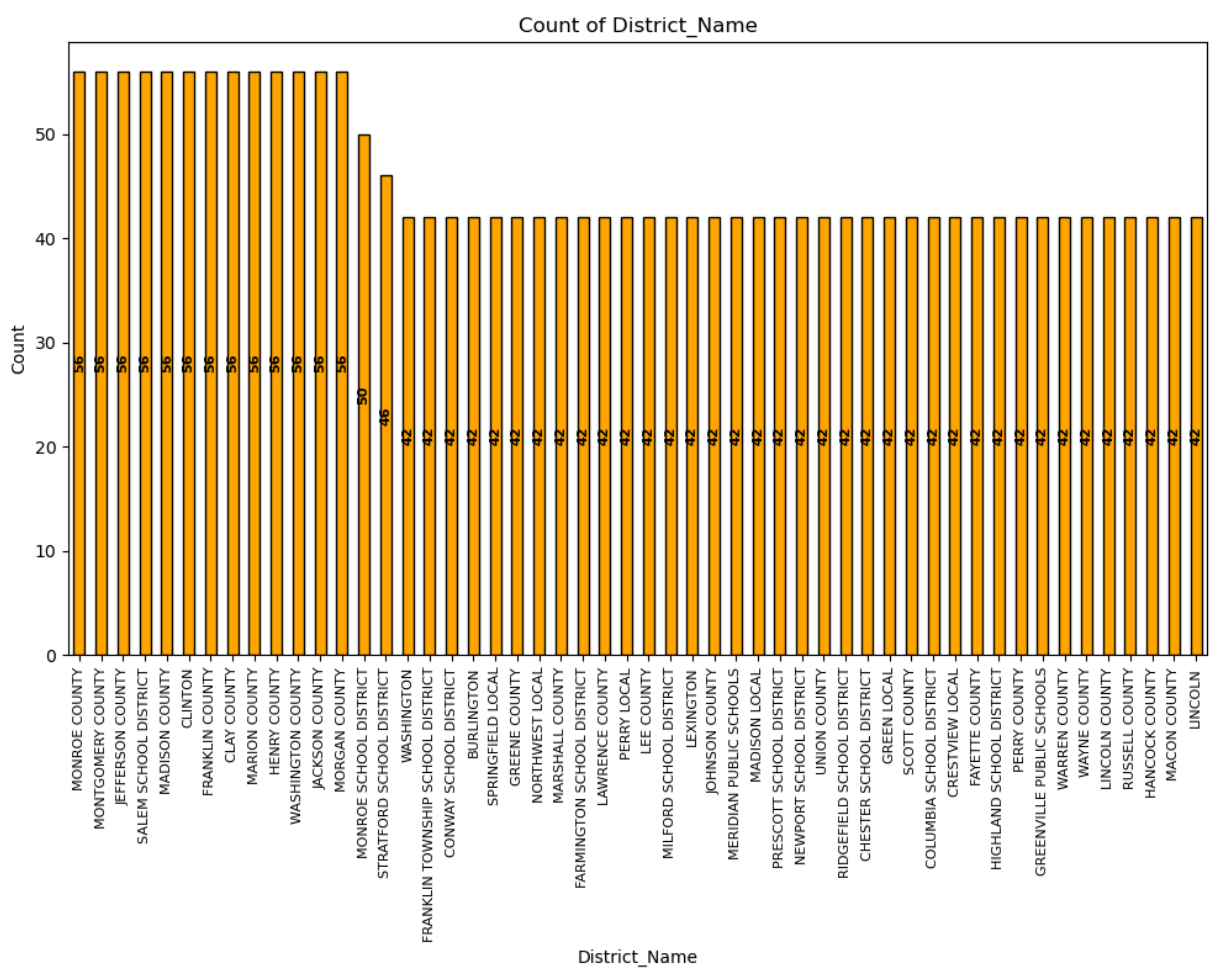
In [15]:
```
districtID_origin = dcd25_renamed.groupby('district_id')[['per_amerindian', 'per_as
ax = districtID_origin.plot(kind='hist', legend=True, figsize=(15, 20), bins=30, al
ax.legend(['American Indian', 'Asian', 'Black', 'White', 'Hispanic', 'Multiracial',
```

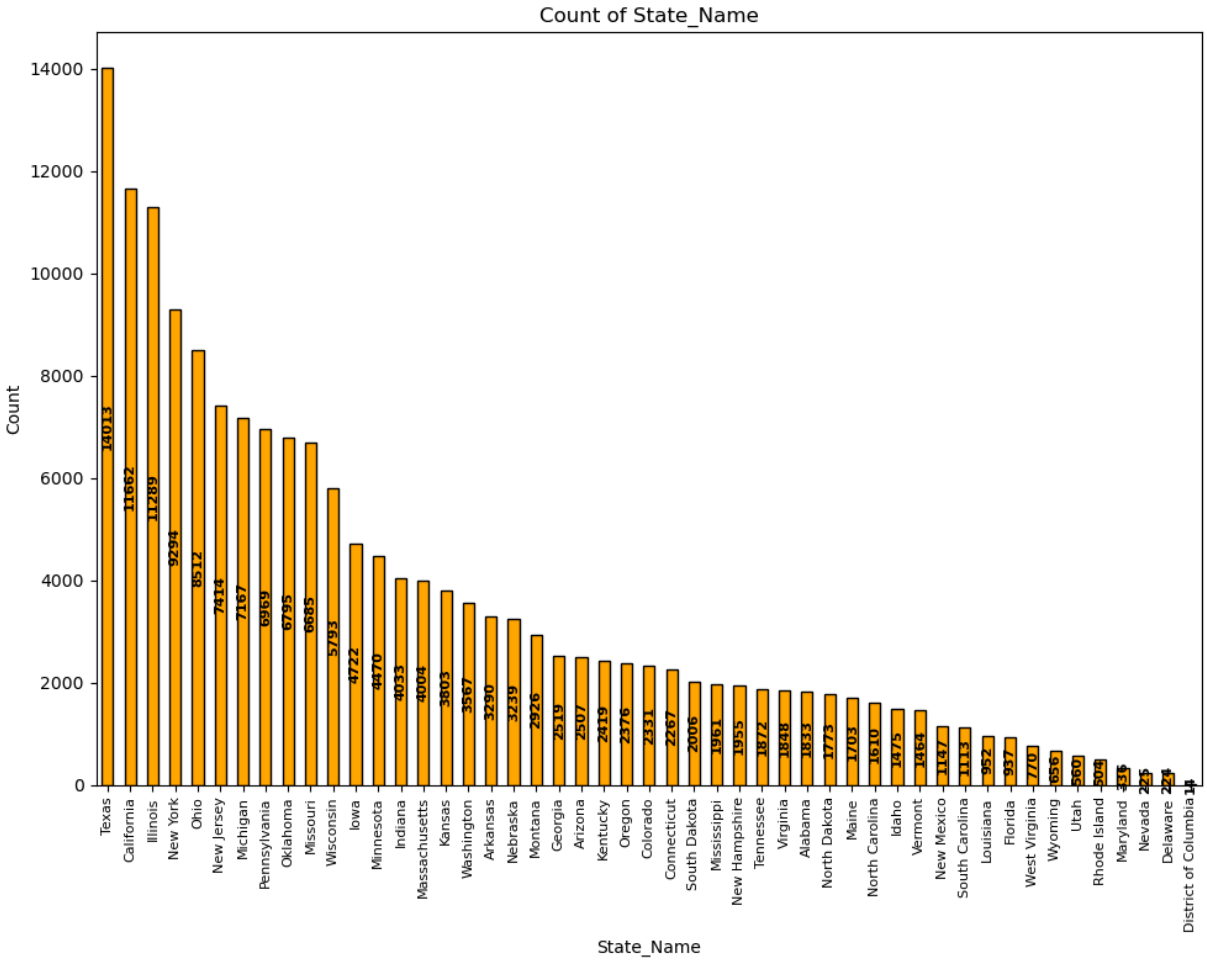


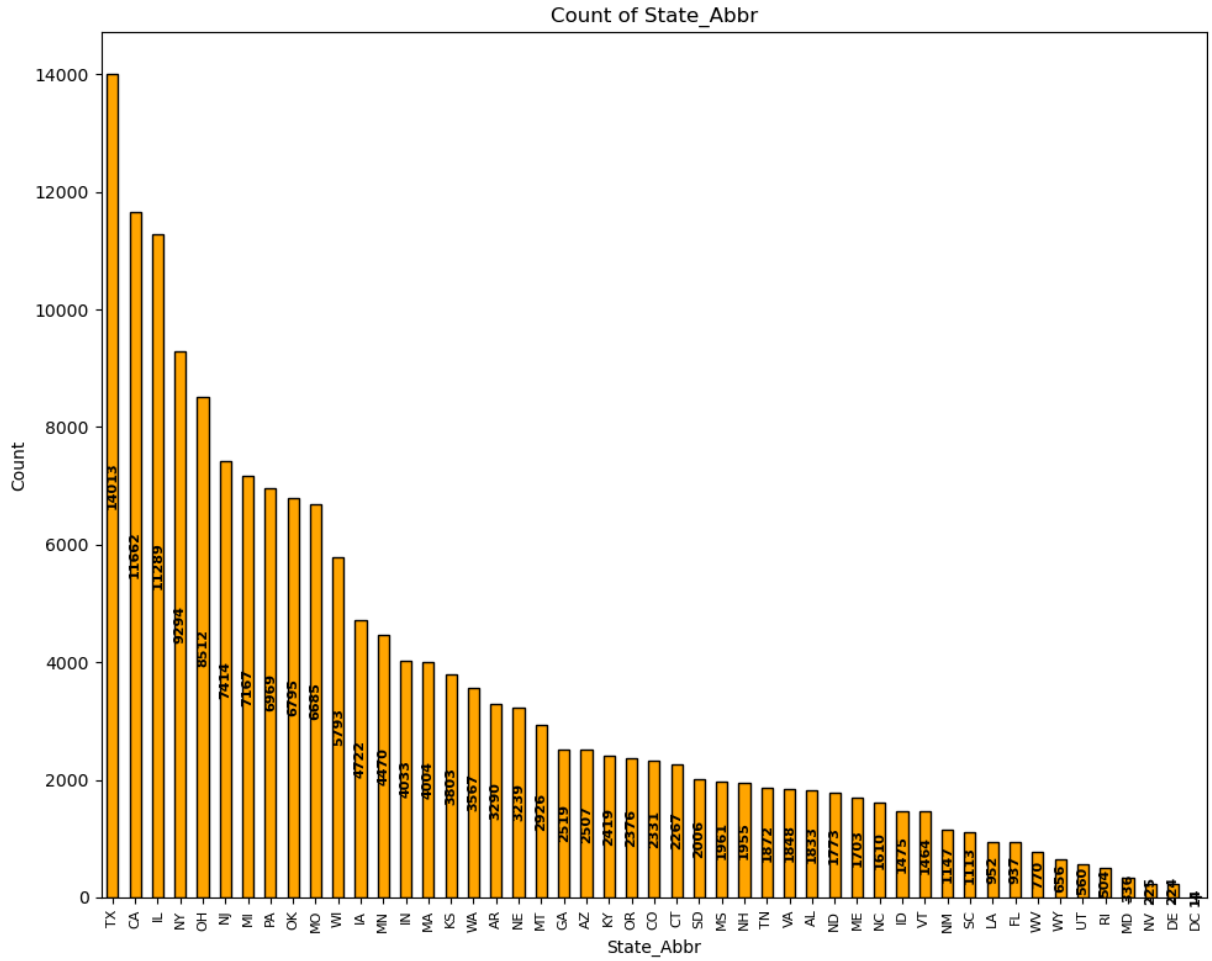
Draw a bar plot to indicate total counts of each categorical variable in a given column.

In [16]:
```
# Bar plot of count of each categorical value
dcd25_rename2 = dcd25.rename(columns={'stabbr': 'State_Abbr', 'state_name': 'State_
categorical_columns = dcd25_rename2.select_dtypes(include=['object', 'category']).c

columns_to_show = 52   # Number of top categories to show in the bar plot
```

```
if not categorical_columns:
    print("No categorical columns found in the dataframe.")
else:
    for col in categorical_columns:
        counts = dcd25_rename2[col].value_counts(dropna=False).head(columns_to_show
        ax = counts.plot(kind='bar', figsize=(10, 8), color='orange', edgecolor='bl
        ax.set_title(f'Count of {col}')
        ax.set_xlabel(col)
        ax.set_ylabel('Count')
        ax.tick_params(axis='x', labelrotation=90, labelsize=8)

        # Add count labels inside bars
        for p in ax.patches:
            height = p.get_height()
            if height > 0:
                ax.text(p.get_x() + p.get_width()/2., height/2,
                        f'{int(height)}',
                        ha='center', va='center', fontsize=8, weight='bold', rotati

        plt.tight_layout()
        plt.show()
```




Count of State_Name
Count
State_Name
0
2000
4000
6000
8000
10000
12000
14000
Texas 14013
California 11662
Illinois 11289
New York 9294
Ohio 8512
New Jersey 7414
Michigan 7167
Pennsylvania 6969
Oklahoma 6795
Missouri 6685
Wisconsin 5793
Iowa 4722
Minnesota 4470
Indiana 4033
Massachusetts 4004
Kansas 3803
Washington 3567
Arkansas 3290
Nebraska 3239
Montana 2926
Georgia 2519
Arizona 2507
Kentucky 2419
Oregon 2376
Colorado 2331
Connecticut 2267
South Dakota 2006
Mississippi 1961
New Hampshire 1955
Tennessee 1872
Virginia 1848
Alabama 1833
North Dakota 1773
Maine 1703
North Carolina 1610
Idaho 1475
Vermont 1464
New Mexico 1147
South Carolina 1113
Louisiana 952
Florida 937
West Virginia 770
Wyoming 656
Utah 560
Rhode Island 504
Maryland 335
Nevada 225
Delaware 224
District of Columbia 11



I do not believe that for this dataset each of these is useful. The stabbr and state_name features should correspond exacty. Although the district count might have some information - like the nuumber of entries for each district, the sheer number of different districts makes what ever may be useful, much less useful.

NOTE: The only way I found to make the plots readable, was to rotate the x axis labels 90 degrees. I believe this is harder to read than no rotation, but the alternative was completely unreadable.

In [17]:
```
dcd25_rename2.select_dtypes(include=['object', 'category']).count()
```

Out[17]:
```
District_Name    171004
State_Name       171004
State_Abbr       171004
dtype: int64
```

one-hot encode the categorical columns

In [18]:
```
dcd25_encoded = pd.get_dummies(dcd25_renamed, drop_first=True)
dcd25_encoded.sample(10)
```

Out[18]:

	year	district_id	actual_spending	req_spending	req_actual_diff	testscore_gap	en
130024	2012	4004110	6991	11424	-4433	0.317608	1
130373	2012	4027300	6462	9594	-3132	-0.196876	1
154799	2010	3904505	9801	9749	52	-0.153132	3
141528	2011	3631950	15053	8173	6880	0.003542	
62875	2017	1802440	8876	12125	-3249	0.217444	1
157084	2010	4834020	11074	11487	-413	0.038497	1
42792	2019	3416440	22812	9884	12928	0.541779	
31471	2020	3626940	20795	7499	13296	NaN	1
1846	2022	1302730	12464	13680	-1216	NaN	3
50565	2018	1732550	8945	7632	1313	0.776355	

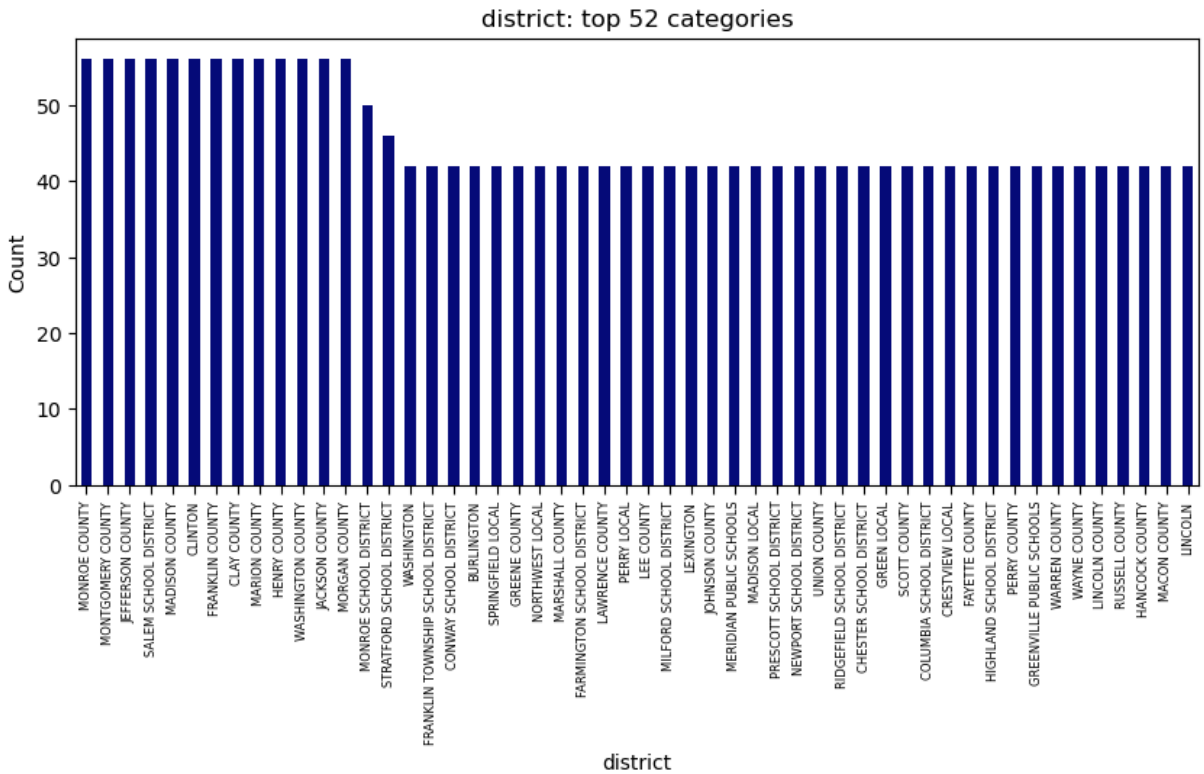
10 rows × 12375 columns

I do not think that one-hot encoding this data was useful in that the categorical column of district name caused the dataset to enlarge exponentially, without a substantial benefit.

Draw other histograms

In [19]:
```
# drop the state abbreviation and state name columns since they are not needed for
dcd25_dropped = dcd25_renamed.drop(columns=['stabbr', 'state_name'])
```

In [20]:
```
##### creat a bar plot of the categorical columns
categorical_columns = dcd25_dropped.select_dtypes(include=["object", "category"]).c

cols_to_display = 52  # Number of top categories to display in the bar plot

if not categorical_columns:
    print("No categorical columns found in dcd25_dropped.")
else:
    for col in categorical_columns:
        counts = dcd25_dropped[col].value_counts(dropna=False).head(cols_to_display
        ax = counts.plot(kind="bar", figsize=(10, 4), color="#070B7A")
        ax.set_title(f"{col}: top {cols_to_display} categories")
        ax.set_xlabel(col)
        ax.set_ylabel("Count")
        ax.tick_params(axis="x", labelrotation=90, labelsize=6, size=4)
        categorical_columns = dcd25_dropped[col]
```



Mean, Median, and Mode

In [21]:
```
numeric_columns = dcd25_renamed.select_dtypes(include=['int64', 'float64']).columns
mydf_numeric = dcd25_renamed[numeric_columns]
mean = mydf_numeric.mean()
median = mydf_numeric.median()
mode = mydf_numeric.mode()

print(f"Mean:\n{mean}\n")
print(f"Median:\n{median}\n")
print(f"Mode:\n{mode}\n")
```

```
Mean:
year               2.015439e+03
district_id        3.011392e+06
actual_spending    1.252683e+04
req_spending       1.202740e+04
req_actual_diff    4.994371e+02
testscore_gap      2.447095e-02
enroll             3.857395e+03
pov                1.708866e-01
per_speced         1.441290e-01
per_eng_learn      4.809390e-02
per_amerindian     2.550963e-02
per_asian          2.147657e-02
per_black          7.219268e-02
per_hispanic       1.438811e-01
per_multi          3.048377e-02
per_pac_hi         1.437527e-03
per_white          7.087118e-01
dtype: float64

Median:
year               2.015000e+03
district_id        3.028770e+06
actual_spending    1.116900e+04
req_spending       1.062600e+04
req_actual_diff    5.045000e+02
testscore_gap      2.577431e-02
enroll             1.267000e+03
pov                1.561093e-01
per_speced         1.407045e-01
per_eng_learn      1.277370e-02
per_amerindian     2.457000e-03
per_asian          6.649600e-03
per_black          1.384875e-02
per_hispanic       5.454550e-02
per_multi          2.236420e-02
per_pac_hi         1.777000e-04
per_white          8.159819e-01
dtype: float64

Mode:
         year  district_id  actual_spending  req_spending  req_actual_diff  \
0      2010.0       100005          10000.0        9100.0            666.0
1         NaN       100006              NaN        9306.0              NaN
2         NaN       100007              NaN           NaN              NaN
3         NaN       100008              NaN           NaN              NaN
4         NaN       100011              NaN           NaN              NaN
...       ...          ...              ...           ...              ...
11628     NaN      5605700              NaN           NaN              NaN
11629     NaN      5605762              NaN           NaN              NaN
11630     NaN      5605830              NaN           NaN              NaN
11631     NaN      5606090              NaN           NaN              NaN
11632     NaN      5606240              NaN           NaN              NaN

       testscore_gap  enroll       pov  per_speced  per_eng_learn  \
0          -0.387226   120.0  0.166667         0.0            0.0
```

```
1          -0.337127   177.0       NaN         NaN            NaN
2          -0.264894     NaN       NaN         NaN            NaN
3          -0.262105     NaN       NaN         NaN            NaN
4          -0.250704     NaN       NaN         NaN            NaN
...              ...     ...       ...         ...            ...
11628            NaN     NaN       NaN         NaN            NaN
11629            NaN     NaN       NaN         NaN            NaN
11630            NaN     NaN       NaN         NaN            NaN
11631            NaN     NaN       NaN         NaN            NaN
11632            NaN     NaN       NaN         NaN            NaN

       per_amerindian  per_asian  per_black  per_hispanic  per_multi  \
0                 0.0        0.0        0.0           0.0        0.0
1                 NaN        NaN        NaN           NaN        NaN
2                 NaN        NaN        NaN           NaN        NaN
3                 NaN        NaN        NaN           NaN        NaN
4                 NaN        NaN        NaN           NaN        NaN
...               ...        ...        ...           ...        ...
11628             NaN        NaN        NaN           NaN        NaN
11629             NaN        NaN        NaN           NaN        NaN
11630             NaN        NaN        NaN           NaN        NaN
11631             NaN        NaN        NaN           NaN        NaN
11632             NaN        NaN        NaN           NaN        NaN

       per_pac_hi  per_white
0             0.0        1.0
1             NaN        NaN
2             NaN        NaN
3             NaN        NaN
4             NaN        NaN
...           ...        ...
11628         NaN        NaN
11629         NaN        NaN
11630         NaN        NaN
11631         NaN        NaN
11632         NaN        NaN

[11633 rows x 17 columns]
```

CONCLUSIONS

Explain what conclusions you would draw from this analysis: are the data what you expect? Are the data likely to be usable? If they are not useable, find some new data!

Conclusions: The data represent a system that has some significant inadequacies. Although the data is, by and large, what I had expected, it was also demoralizing. The data itself is useful.

What is the overall shape of the distribution? Is it normal, skewed, bimodal, uniform, etc.?

In [22]:
```
dcd25_renamed.shape
```

Out[22]:
```
(171004, 20)
```

In [23]:
```
# The distributions are predominantly right-skewed and not normal. The racial demog
```

Are there any outliers present? (Data points that are far from the others.)

In [24]:
```
# find outliers in the numeric columns using IQR method
Q1 = mydf_numeric.quantile(0.25)
Q3 = mydf_numeric.quantile(0.75)
IQR = Q3 - Q1
outliers = ((mydf_numeric < (Q1 - 1.5 * IQR)) | (mydf_numeric > (Q3 + 1.5 * IQR))).
print(f"Number of outliers in each numeric column:\n{outliers}")
```

```
Number of outliers in each numeric column:
year                   0
district_id            0
actual_spending     8531
req_spending        9434
req_actual_diff    12197
testscore_gap       2292
enroll             17748
pov                 2713
per_speced          3784
per_eng_learn      20093
per_amerindian     23105
per_asian          21496
per_black          26151
per_hispanic       20460
per_multi           7755
per_pac_hi         14959
per_white              0
dtype: int64
```

If there are multiple related histograms, how does the distribution change across different groups?

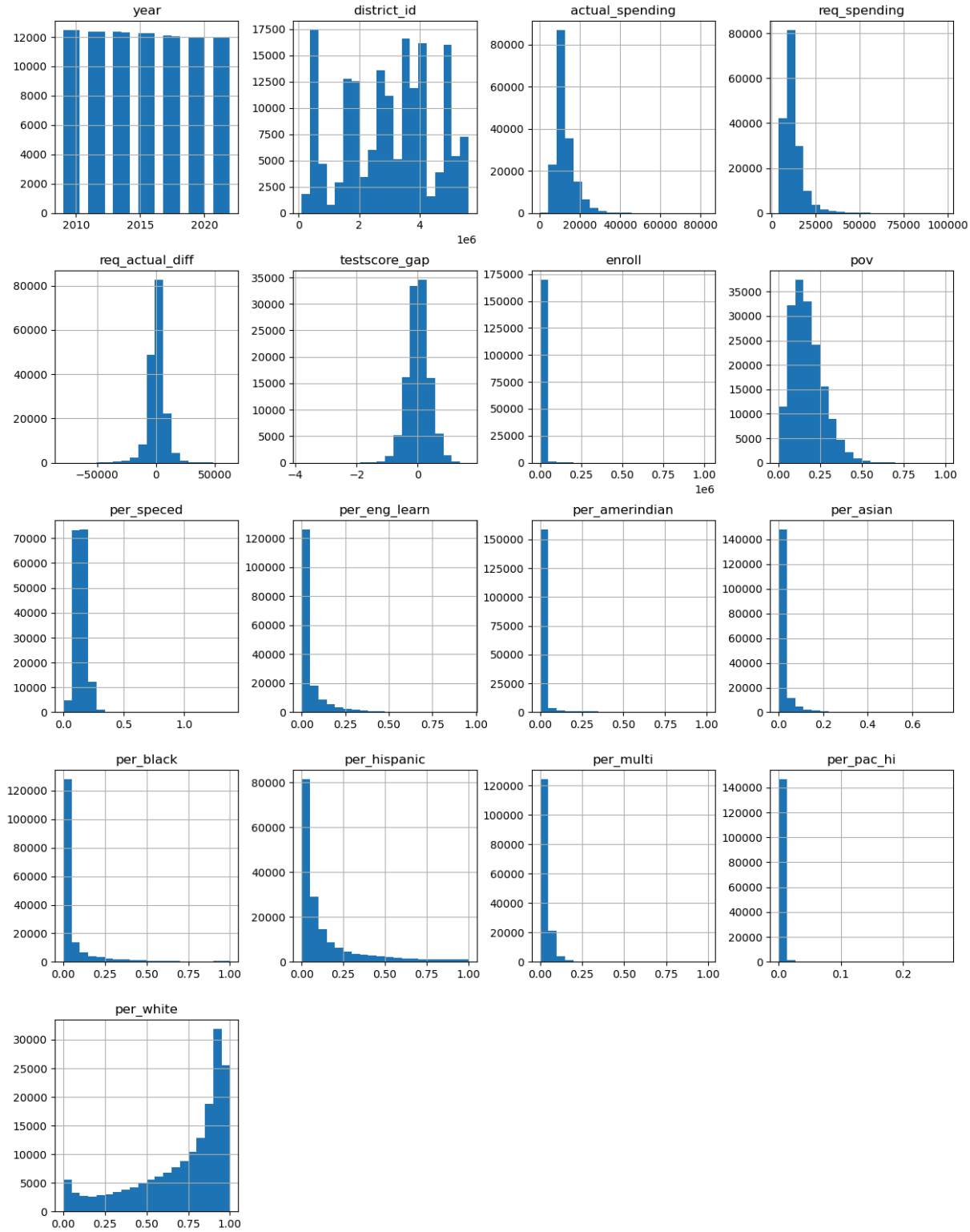
What are the minimum and maximum values represented in each histogram?

In [25]:
```
# iterate through the dataset printing out the minimum and maximum values of each n
for col in numeric_columns:
    col_min = dcd25_renamed[col].min()
    col_max = dcd25_renamed[col].max()
    feature = col.replace('_', ' ').title()  # Convert column name to a more readab
    feature = feature.replace('Per ', 'Percentage of ').replace('Req ', 'Required '
    print(f"{feature}:     min = {col_min},     max = {col_max}")
```

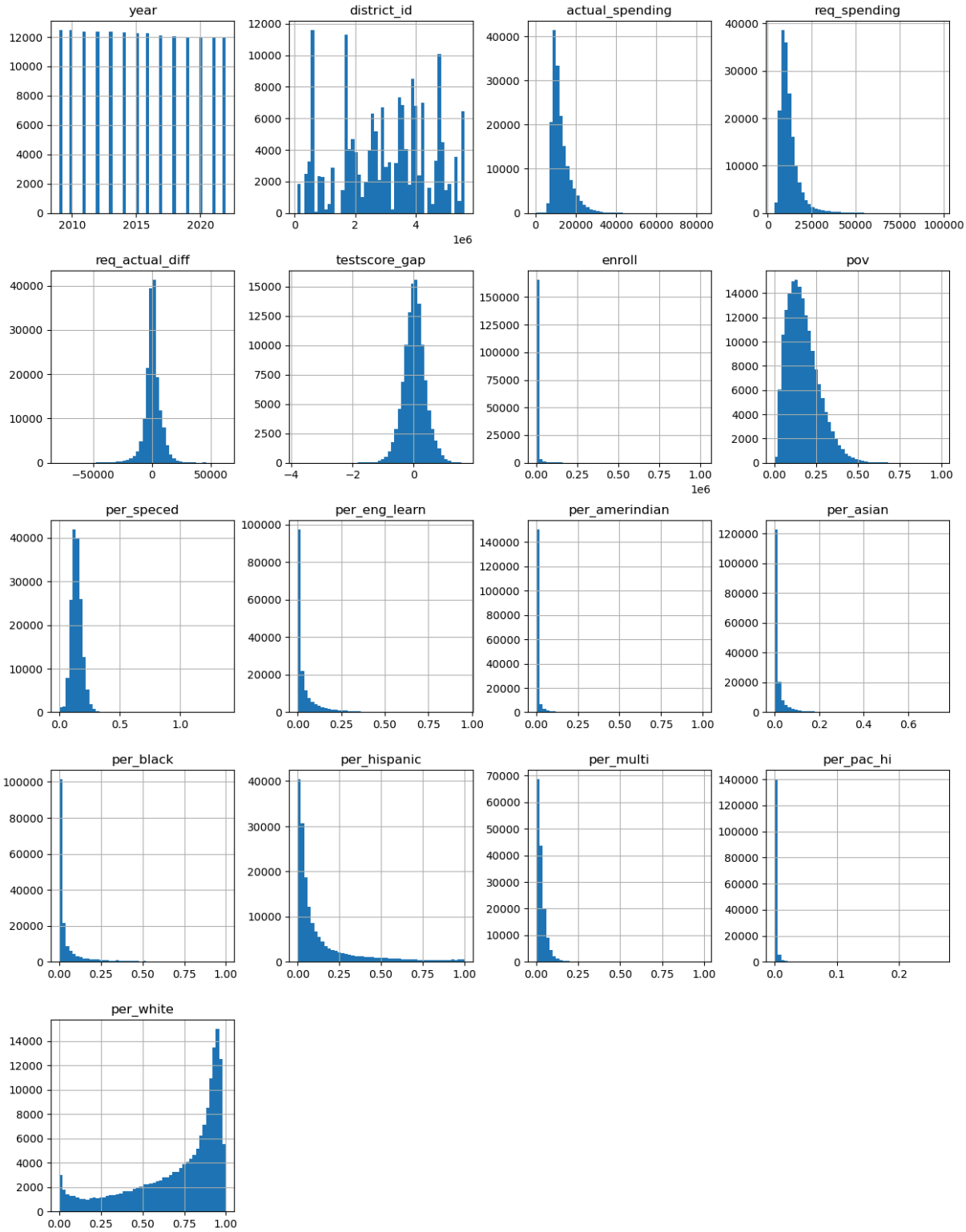
```
Year:     min = 2009,     max = 2022
District Id:     min = 100005,     max = 5606240
Actual Spending:     min = 251,     max = 83114
Required Spending:     min = 3802,     max = 98479
Required Actual Difference:     min = -78683,     max = 62435
Testscore Gap:     min = -3.7977975,     max = 1.6575414
Enroll:     min = 100,     max = 1014020
Poverty:     min = 0.0,     max = 1.0
Percentage of Special Education Students:     min = 0.0,     max = 1.379532
Percentage of English Learner Students:     min = 0.0,     max = 0.9567404
Percentage of American Indian Students:     min = 0.0,     max = 1.0
Percentage of Asian Students:     min = 0.0,     max = 0.7474986
Percentage of Black Students:     min = 0.0,     max = 1.0
Percentage of Hispanic Students:     min = 0.0,     max = 1.0
Percentage of Multiracial Students:     min = 0.0,     max = 0.9902912
Percentage of Pacific Islander \ Hawaiian Students:     min = 0.0,     max = 0.26749
61
Percentage of  White Students:     min = 0.0,     max = 1.0
```

How do bin sizes affect the histogram? Does changing the bin width reveal different patterns in the data?

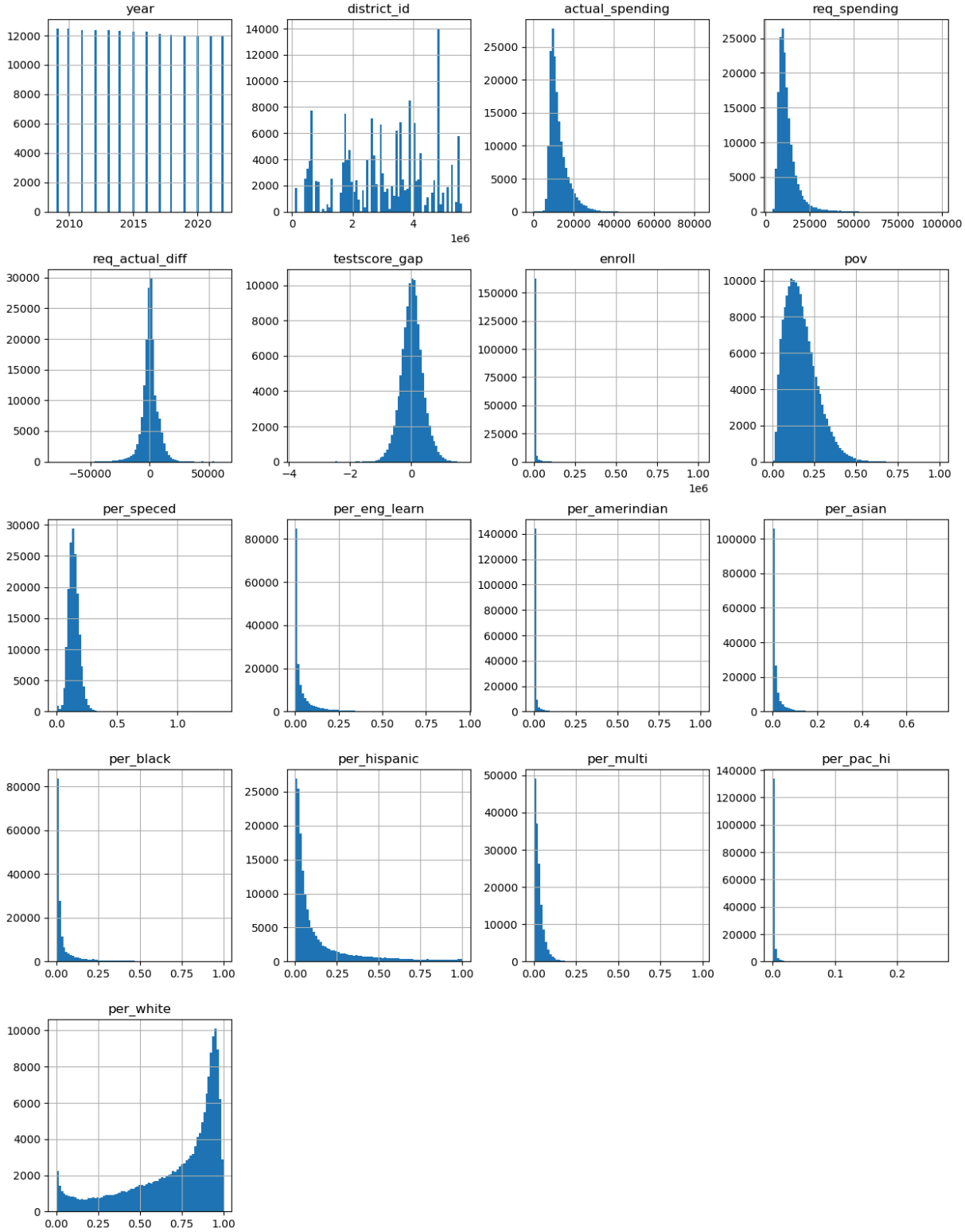
In [26]:
```
dcd25_renamed.hist(figsize=(15, 20), bins=20);
```



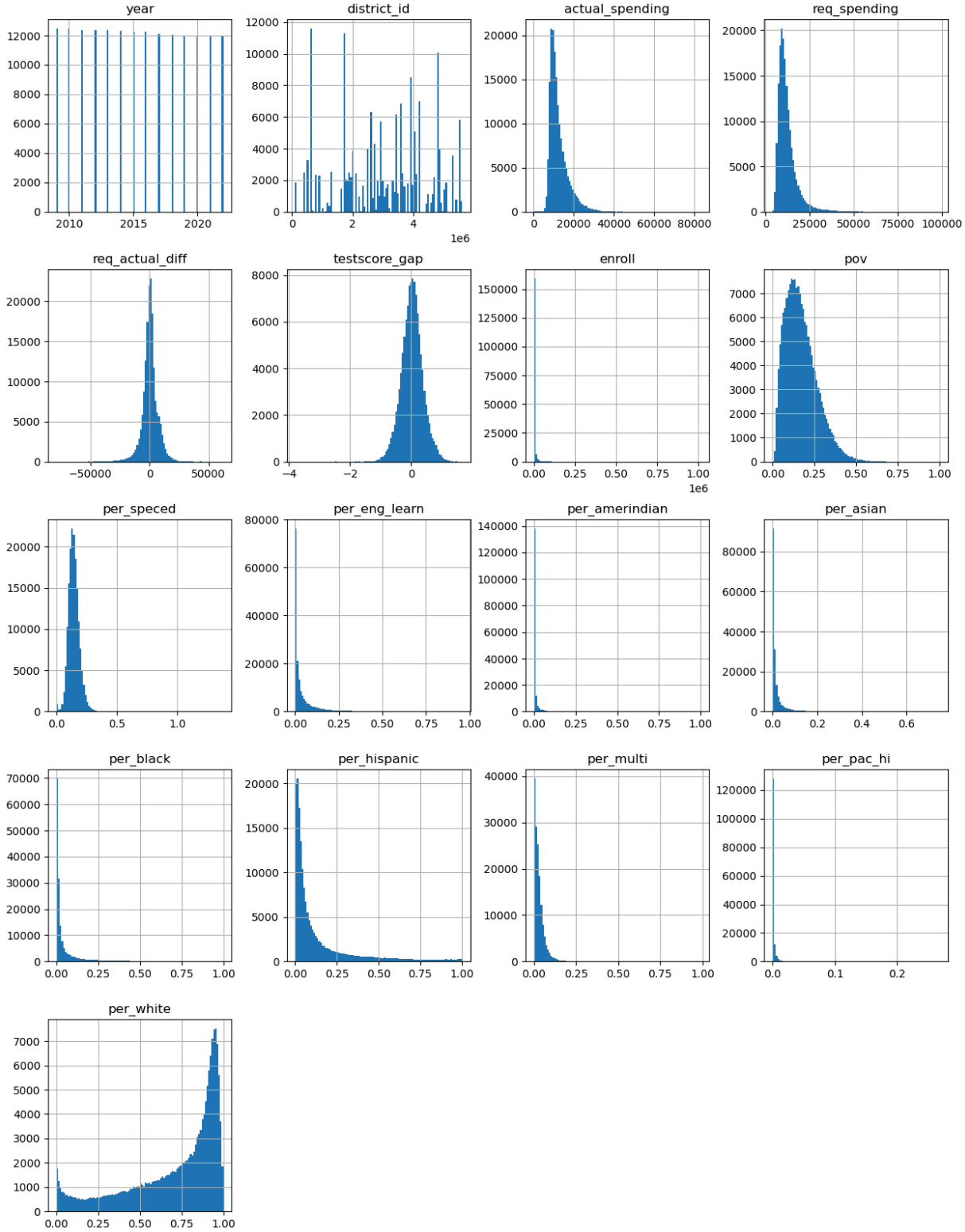
In [27]:
```
dcd25_renamed.hist(figsize=(15, 20), bins=50);
```



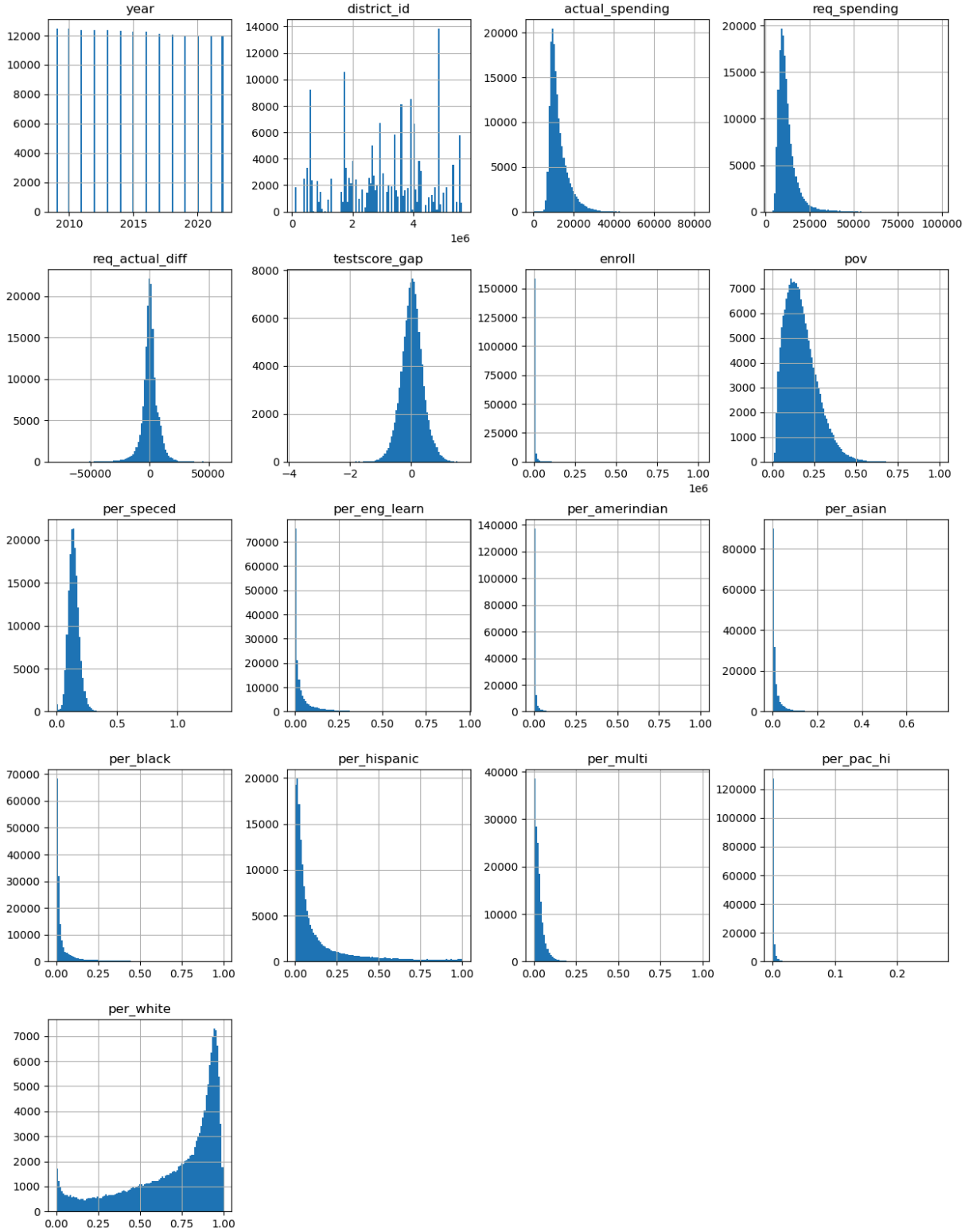
In [28]:
```
dcd25_renamed.hist(figsize=(15, 20), bins=75);
```



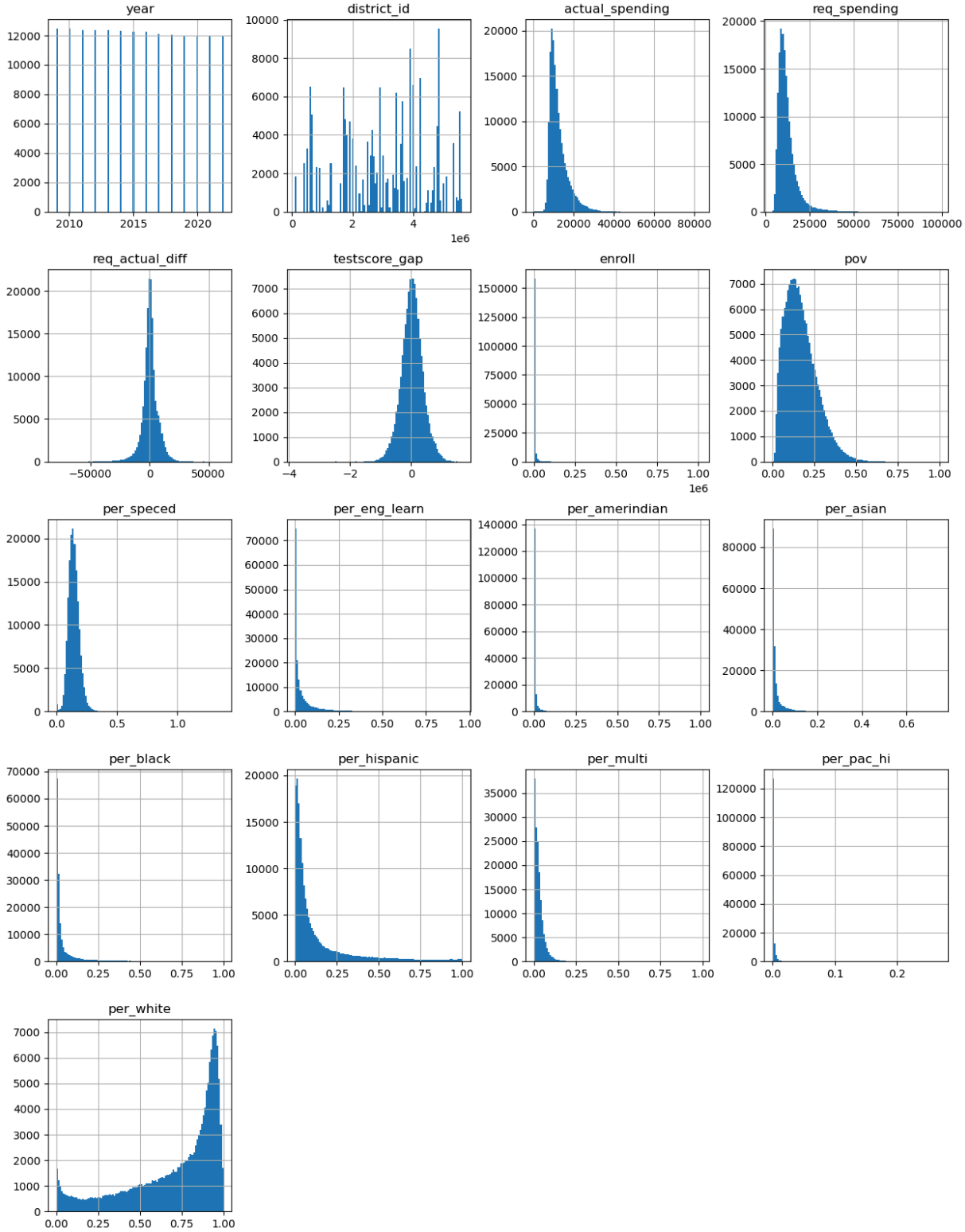
In [29]:
```
dcd25_renamed.hist(figsize=(15, 20), bins=100);
```



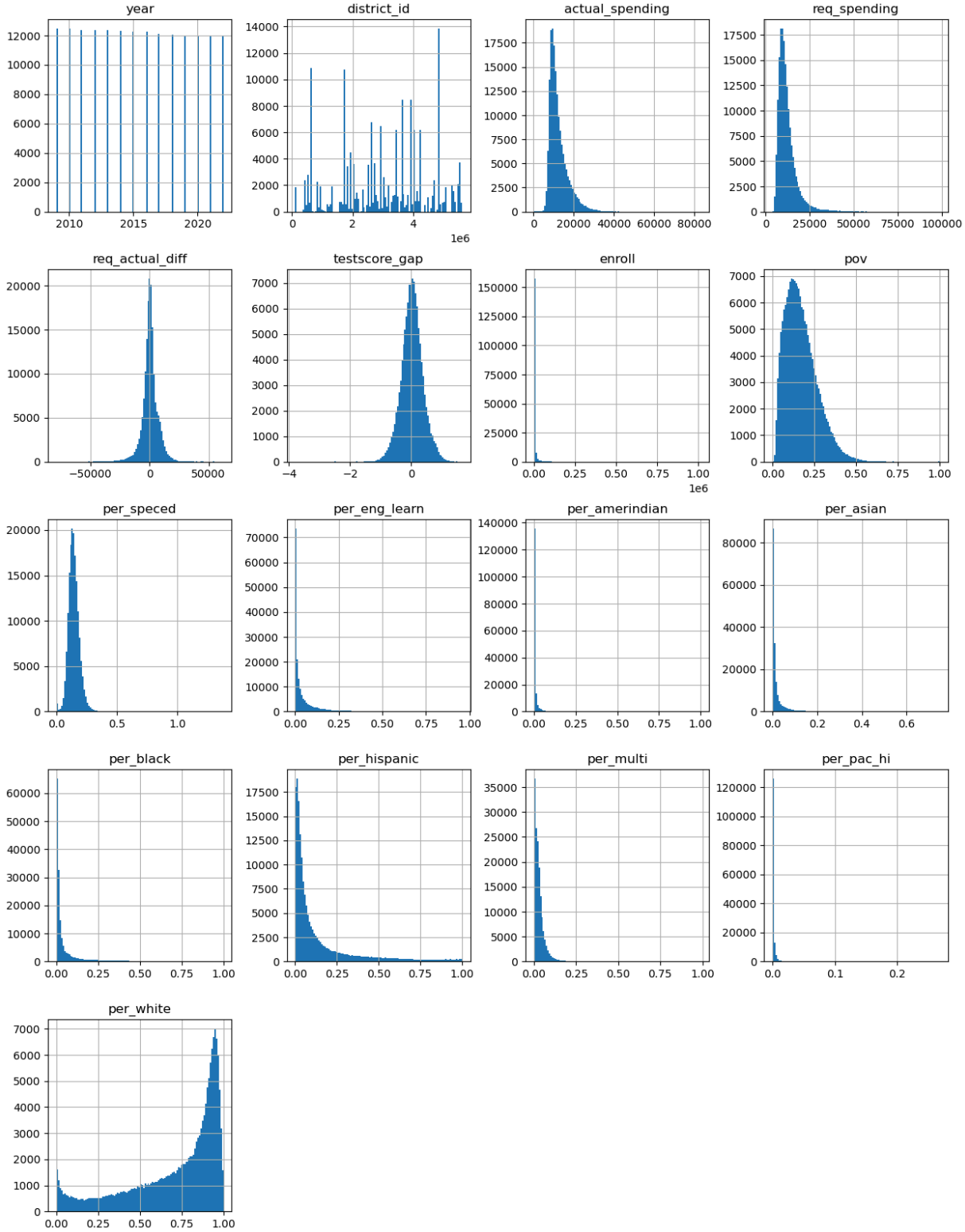
In [30]:
```
dcd25_renamed.hist(figsize=(15, 20), bins=103);
```



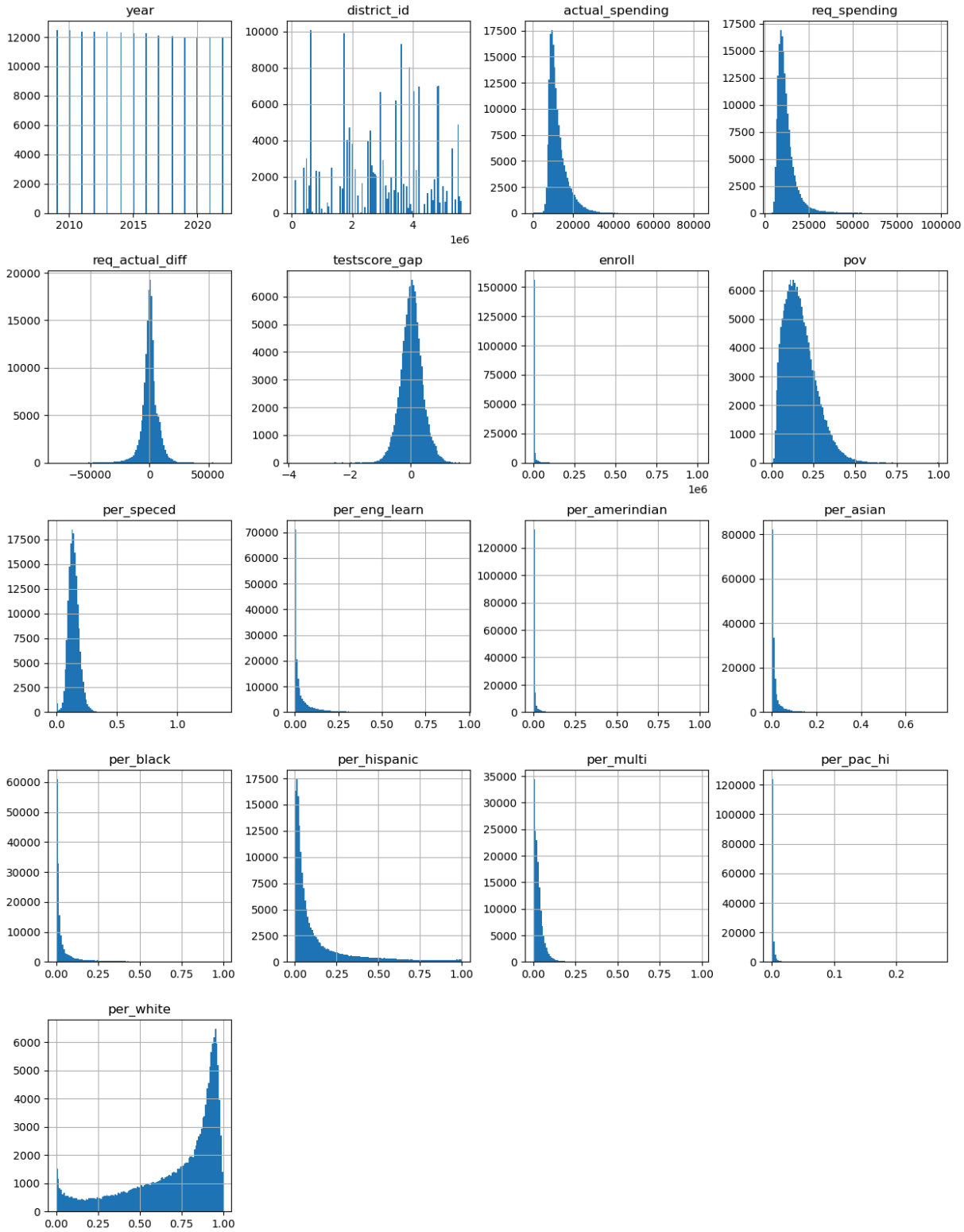
In [31]:
```
dcd25_renamed.hist(figsize=(15, 20), bins=105);
```



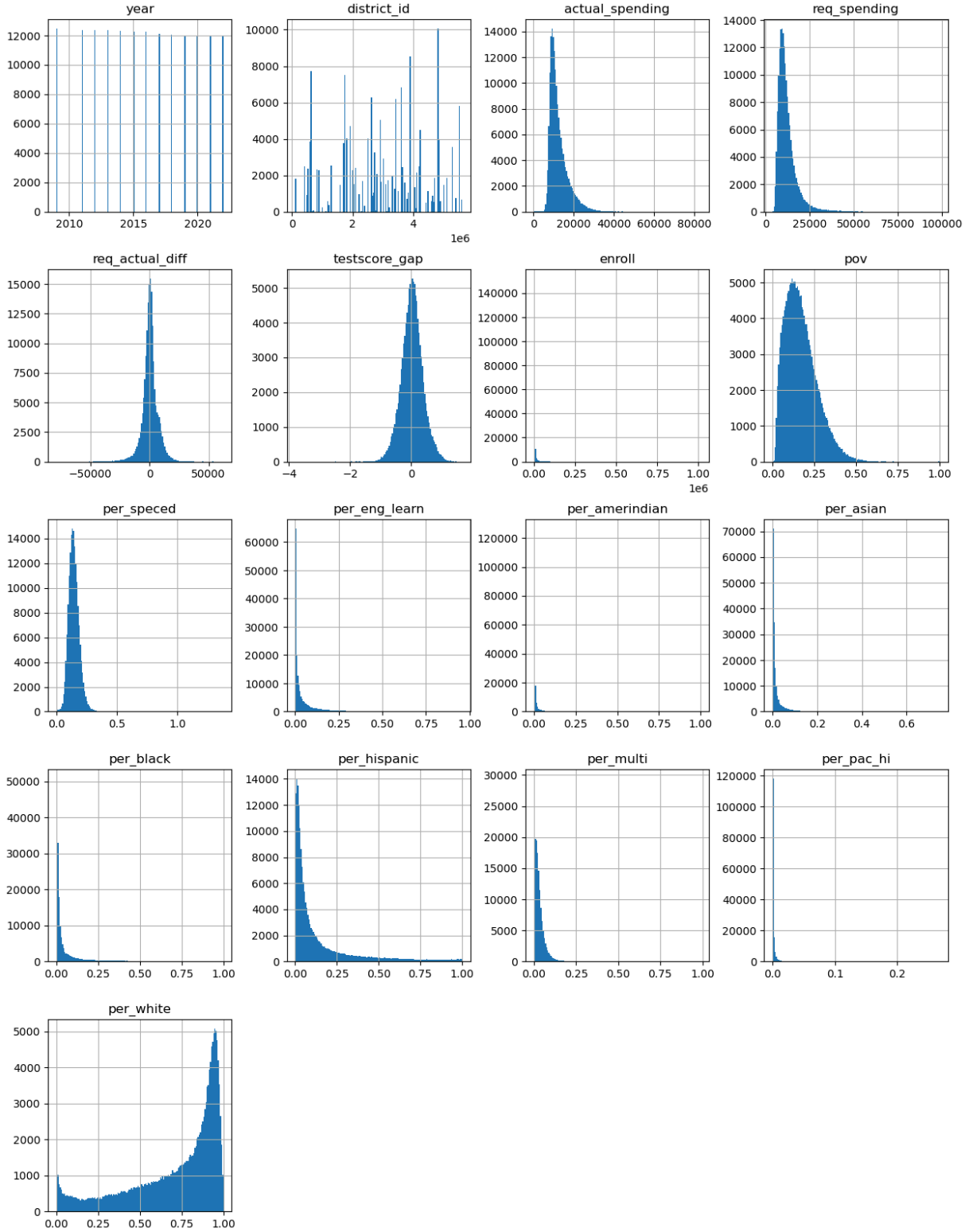
In [32]:
```
dcd25_renamed.hist(figsize=(15, 20), bins=110);
```



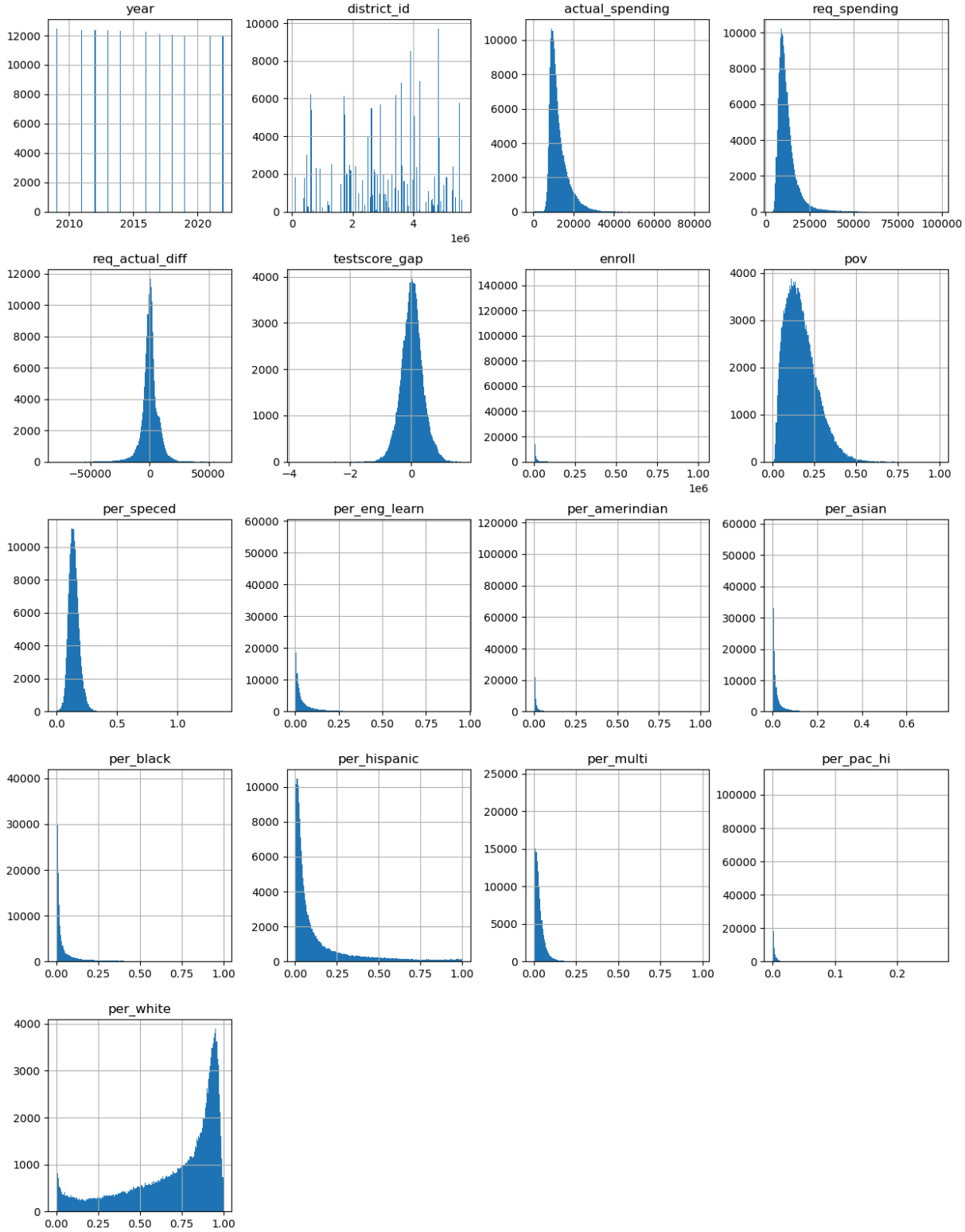
In [33]:
```
dcd25_renamed.hist(figsize=(15, 20), bins=120);
```



In [34]:
```
dcd25_renamed.hist(figsize=(15, 20), bins=150);
```



In [35]:
```
dcd25_renamed.hist(figsize=(15, 20), bins=200);
```



The bin size seems to make the histograms more exact in that there seems to be a better fit to the data. Although the number of bins do not seem to make a difference after about 100 (this may have to do with the fact that there are only 100 states).

Does the distribution appear normal, or does it have a different distribution?

In [36]:
```
# Check skewness and kurtosis for each numeric column to assess if it is normally d
print("Distribution Analysis:\n")
print("=" * 80)
for col in dcd25_renamed.select_dtypes(include=['number']).columns:
    skew = dcd25_renamed[col].skew()
    kurt = dcd25_renamed[col].kurtosis()
    print(f"\n{col}:")
    print(f"  Skewness: {skew:.4f} {'(approximately symmetric)' if abs(skew) < 0.5
    print(f"  Kurtosis: {kurt:.4f} {'(approximately normal)' if abs(kurt) < 1 else

print("\n" + "=" * 80)
print("\nNote: For a normal distribution:")
print("  - Skewness should be close to 0 (between -0.5 and 0.5)")
print("  - Kurtosis should be close to 0 (between -1 and 1)")
```

```
Distribution Analysis:

================================================================================

year:
  Skewness: 0.0198 (approximately symmetric)
  Kurtosis: -1.2103 (light-tailed)

district_id:
  Skewness: -0.1655 (approximately symmetric)
  Kurtosis: -0.9720 (approximately normal)

actual_spending:
  Skewness: 2.0953 (right-skewed)
  Kurtosis: 8.5803 (heavy-tailed)

req_spending:
  Skewness: 2.8622 (right-skewed)
  Kurtosis: 14.7727 (heavy-tailed)

req_actual_diff:
  Skewness: -0.5943 (left-skewed)
  Kurtosis: 6.1489 (heavy-tailed)

testscore_gap:
  Skewness: -0.0434 (approximately symmetric)
  Kurtosis: 0.8615 (approximately normal)

enroll:
  Skewness: 32.6248 (right-skewed)
  Kurtosis: 1707.0579 (heavy-tailed)

pov:
  Skewness: 0.8678 (right-skewed)
  Kurtosis: 0.8382 (approximately normal)

per_speced:
  Skewness: 0.8625 (right-skewed)
  Kurtosis: 7.9881 (heavy-tailed)

per_eng_learn:
  Skewness: 3.4304 (right-skewed)
  Kurtosis: 14.9761 (heavy-tailed)

per_amerindian:
  Skewness: 6.5164 (right-skewed)
  Kurtosis: 47.8780 (heavy-tailed)

per_asian:
  Skewness: 6.0221 (right-skewed)
  Kurtosis: 49.5437 (heavy-tailed)

per_black:
  Skewness: 3.5091 (right-skewed)
  Kurtosis: 13.3585 (heavy-tailed)
```

```
per_hispanic:
  Skewness: 2.1976 (right-skewed)
  Kurtosis: 4.4324 (heavy-tailed)

per_multi:
  Skewness: 5.6260 (right-skewed)
  Kurtosis: 85.4898 (heavy-tailed)

per_pac_hi:
  Skewness: 15.7775 (right-skewed)
  Kurtosis: 459.4449 (heavy-tailed)

per_white:
  Skewness: -1.0858 (left-skewed)
  Kurtosis: 0.0943 (approximately normal)

================================================================================

Note: For a normal distribution:
  - Skewness should be close to 0 (between -0.5 and 0.5)
  - Kurtosis should be close to 0 (between -1 and 1)
```

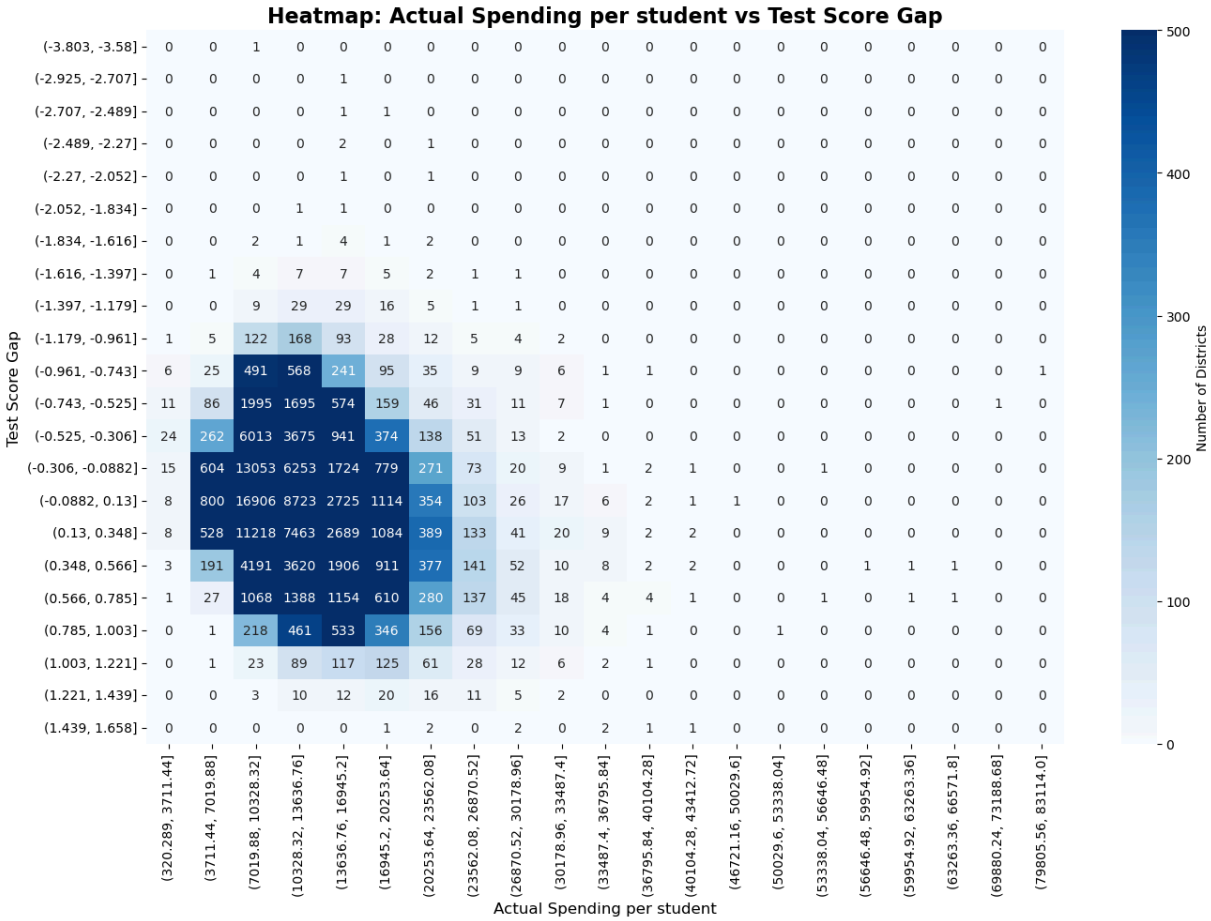
Heatmap

Although I am not sold on this type of graph for presenting the data, I felt it would be helpful to at least try it out.

In [98]:

```
################################################################################
# set initial parameters for the heatmap such as the number of bins for categorizin
# the data, the columns to compare, the titles etc. These parameters will be used
# to create a heatmap that compares the actual spending per student with the test
# score gap across different districts, allowing us to visualize any potential rela
# or patterns between these two features.

# I set the sample number to 25 to create a heatmap with 25 bins for both the actua
# spending per student and the test score gap. This allows for a detailed compariso
# between the two features while still maintaining a manageable number of categorie
# for visualization. By using 25 bins, we can capture the variability in the data a
# identify any patterns or trends that may exist between actual spending and test s
# across different districts.
sample_number = 25
# The first column/feature I want to compare is 'actual_spending', which represents
# amount of money spent per student in each district. This column is crucial for ou
# as it allows us to examine how the actual spending correlates with the test score
# is a key outcome variable in our study. By comparing actual spending with the tes
# we can gain insights into whether higher spending is associated with smaller gaps
# which can inform policy decisions and resource allocation in education.
Column_to_compare1 = 'actual_spending'
# set the first title to 'Actual Spending per student' to clearly indicate that the
# the actual spending per student across different districts. This title helps to c
# and makes it easier for viewers to understand what the heatmap represents, especi
# to the second column, which is the test score gap.
title1 = 'Actual Spending per student'
# The second column/feature I want to compare is 'testscore_gap', which represents
# between different groups of students
```

```
Column_to_compare2 = 'testscore_gap'
# The title for the second column is set to 'Test Score Gap' to clearly indicate th
# the actual spending per student with the gap in test scores, which is a key outco
# analysis. This title helps to contextualize the heatmap and makes it easier for v
# what the data represents.
title2 = 'Test Score Gap'
# I use the Set1_r color palette from seaborn to create a visually appealing heatma
# relationship between actual spending per student and test score gap across differ
# heatmap will help identify patterns and trends in the data, such as whether highe
# associated with smaller test score gaps.
color_map = sns.color_palette("Blues", as_cmap=True)
# Create a new DataFrame with only the two columns to compare and drop rows with mi
df = dcd25_renamed[[Column_to_compare1, Column_to_compare2]].dropna()  # Drop rows
# set the minimum value for the heatmap to 0 to ensure that the color scale starts
# allows for a clearer visualization of the differences in the number of districts
# of actual spending and test score gaps. This helps to highlight areas with higher
# of districts and makes it easier to interpret the heatmap.
minimum_value = 0
# set the maximum value for the heatmap to 500 to cap the color scale at a reasonab
# prevents extreme values from skewing the visualization and allows for a more mean
# of the number of districts across different bins of actual spending and test scor
# to ensure that the heatmap effectively highlights patterns and trends in the data
# dominated by outliers.
maximum_value = 500
################################################################################

# Create bins for spending difference and test score gap
feature1 = pd.cut(df[Column_to_compare1], bins=sample_number)
feature2 = pd.cut(df[Column_to_compare2], bins=sample_number)

# Create a pivot table showing the count of districts for each combination
heatmap_data = pd.crosstab(feature2, feature1)

# Create the heatmap
fig, ax = plt.subplots(figsize=(14, 10))
sns.heatmap(heatmap_data, annot=True, fmt='d', cmap=color_map,
            vmin=minimum_value, vmax=maximum_value,
            cbar_kws={'label': 'Number of Districts'}, ax=ax)

# optionally Add legend text
# ax.text(0.9, 1.03, f'Color Scale Range: {minimum_value} - {maximum_value} distric
#         transform=ax.transAxes, fontsize=10, verticalalignment='top',
#         bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

plt.title(f'Heatmap: {title1} vs {title2}', fontsize=16, fontweight='bold')
plt.xlabel(title1, fontsize=12)
plt.ylabel(title2, fontsize=12)
plt.tight_layout()
plt.show()
plt.close()
```



4. Storytelling With Data graph

Reproduce any graph of your choice in p. 52-68 of the Storytelling With Data book as best you can. (The second half of chapter two). You do not have to get the exact data values right, just the overall look and feel.

Create survey dataframe from data used to create graph in SWD

In [37]:
```
survey_index = [
    "Survey item A",
    "Survey item B",
    "Survey item C",
    "Survey item D",
    "Survey item E",
]

survey_data = {
    "Strongly Agree": [33, 20, 20, 17, 9],
    "Agree": [27, 28, 15, 23, 18],
    "Neutral": [30, 35, 45, 20, 32],
    "Disagree": [5, 11, 14, 32, 25],
    "Strongly Disagree": [5, 6, 6, 8, 16],
```

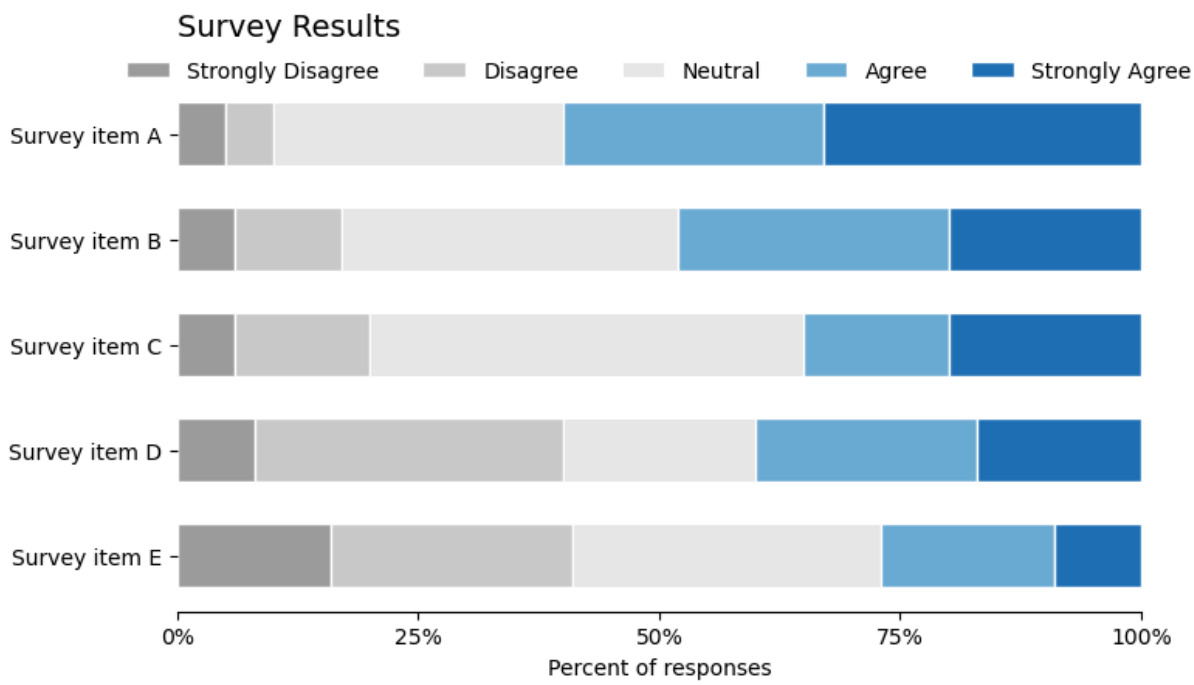
```
    "Total": [100, 100, 100, 100, 100],
}

survey_df = pd.DataFrame(survey_data, index=survey_index)
survey_df
```

Out[37]:

	Strongly Agree	Agree	Neutral	Disagree	Strongly Disagree	Total
Survey item A	33	27	30	5	5	100
Survey item B	20	28	35	11	6	100
Survey item C	20	15	45	14	6	100
Survey item D	17	23	20	32	8	100
Survey item E	9	18	32	25	16	100

Recreate Graph on page 59

In [38]:

```
# create the likert titles
likert_order = [
    "Strongly Disagree",
    "Disagree",
    "Neutral",
    "Agree",
    "Strongly Agree",
]

# make a copy of the survey dataframe with only the likert columns
plot_df = survey_df[likert_order].copy()

# set colors for the likert categories
colors = {
    "Strongly Disagree": "#9e9e9e",
    "Disagree": "#c9c9c9",
    "Neutral": "#e6e6e6",
    "Agree": "#6baed6",
    "Strongly Agree": "#2171b5",
}

# create the figure and axis for the plot
fig, ax = plt.subplots(figsize=(8, 4.6))

# Plot stacked bars from left to right (0 to 100)
left = 0
for col in likert_order:
    ax.barh(
        plot_df.index,
        plot_df[col],
        left=left,
        color=colors[col],
        edgecolor="white",
        height=0.6,
        label=col,
```

```
    )
    left += plot_df[col]

ax.set_xlim(0, 100)
ax.set_xticks([0, 25, 50, 75, 100])
ax.set_xticklabels(["0%", "25%", "50%", "75%", "100%"])
ax.set_xlabel("Percent of responses")
ax.set_ylabel("")

# set the survey title with padding and font size, aligned to the left
ax.set_title("Survey Results", pad=20, fontsize=14, loc="left")
ax.invert_yaxis()

ax.legend(ncol=5, frameon=False, loc="upper center", bbox_to_anchor=(0.5, 1.07))

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_visible(False)

plt.tight_layout()
plt.show()
```